

Real Time Video Communication for Sign and Non-Sign Users

By

Hammad Ali

(Reg. 2022-GCUF-02725)

Muhammad Muneeb

(Reg. 2022-GCUF-02724)

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION TECHNOLOGY
GOVERNMENT COLLEGE UNIVERSITY, FAISALABAD.

June 2026

DECLARATION

The work reported in this documentation was carried out by us under the supervision of **Mam. Rabia Saleem** Department of Information Technology Government College University, Faisalabad, Pakistan.

We hereby declare that the title of project **Real Time Video Communication for Sign and Non-Sign Users** and the contents of the documentation are the product of my own work and no part has been copied from any published source (except the references, standard mathematical or genetic models /equations /formulas/protocols etc.) We further declare that this work has not been submitted for award of any other degree /diploma. The University may take action if the information provided is found inaccurate at any stage.

Signature of the Student

Hammad Ali

2022-GCUF-02725

Signature of the Student

Muhammad Muneeb

2022-GCUF-02724

CERTIFICATE BY SUPERVISORY COMMITTEE

We certify that the contents and form of documentation submitted by Mr. Hammad Ali Registration No: 2022-GCUF-02725 and by Mr. Muneeb Registration No: 2022-GCUF-02724. Has been Found satisfactory and in accordance with the prescribed format. We recommend it to be processed for the evaluation by the External Examiner for the award of Degree.

Signature of Supervisor

Name.....

Designation with stamp.....

Member of Supervisory Committee

Signature

Name.....

Designation with stamp.....

Member of Supervisory Committee

Signature

Name.....

Designation with stamp.....

Chairperson

Signature

Name.....

Designation with stamp.....

DEDICATED

TO

Our Parents

We dedicate this project to our beloved parents, whose endless prayers, patience, and sacrifices have been our greatest source of strength and motivation throughout this journey. Their unconditional love and constant encouragement have guided us at every step. We also dedicate this work to our respected supervisor, Mam Rabia Saleem, whose expert guidance and support made this project possible. Finally, we dedicate this effort to all hearing-impaired and speech-impaired individuals, with the hope that technology like Sign-Bridge will one day help bridge the communication gap they face every day.

ACKNOWLEDGEMENTS

All praise to ALMIGHTY ALLAH, the most merciful and the most compassionate and His Holy Prophet MUHAMMAD (Peace Be Upon Him), the most perfect and exalted among all, who is forever a torch of guidance and knowledge for humanity as a whole.

We would like to express our deepest gratitude to our supervisor, Mam Rabia Saleem, Department of Information Technology, Government College University Faisalabad, for her invaluable guidance, continuous support, and constructive feedback throughout the development of this project. Her expertise and encouragement were instrumental in shaping the direction and quality of this work.

We are also grateful to the Department of Information Technology and the faculty members who provided us with the knowledge and skills necessary to undertake this project. Special thanks to our friends and colleagues for their moral support and cooperation.

Finally, we thank our families for their patience, prayers, and unwavering support during the entire duration of this project.

Hammad Ali
Muhammad Muneeb

Contents

Chapter 1	13
1.1. Introduction	13
1.2. Project Objectives.....	13
1.2.1. General Objective	14
1.2.2. Specific Objectives	14
1.3. Scope of the System.....	15
1.4. Technical Feasibility	16
1.5. Economic Feasibility.....	17
1.6. Operational Feasibility	17
1.7. Stakeholders	18
1.7.1. Hearing-Impaired or Speech-Impaired Users.....	18
1.7.2. Non-Sign Language Users.....	18
1.7.3. Students and Researchers	18
1.7.4. Project Supervisor and Evaluation Committee.....	19
1.7.5. Educational Institutions	19
1.7.6. Developers and Maintainers	19
1.8. User of the System.....	19
1.8.1. Sign Language User	19
1.8.2. Non-Sign Language User	19
1.8.3. General Communication User.....	20
1.8.4. Developer or Administrator	20
1.9. Functional Requirements	20
Description of Each Requirement in Requirement Shell	20
1.9.1. Join Call Room	20
1.9.2. Access Camera.....	21
1.9.3. Establish Video Call.....	22
1.9.4. Connect To Signaling Server	23
1.9.5. Detect Hand Marks	23
1.9.6. Recognize ASL Signs	24
1.9.7. Build Text From Recognized Signs.....	25
1.9.8. Leave Room Call.....	26

1.9. Non Functional Requirements	27
1.10.1. Performance	27
1.10.2. Usability	27
1.10.3. Reliability.....	27
1.10.4. Accessibility	28
1.10.5. Compatibility	28
1.10.6. Security.....	28
1.10.7. Maintainability	28
1.10.8. Scalability	28
1.10.9. Accuracy	28
1.11. Schedule of Project	29
Chapter 2	31
2.1. Use Case Model.....	31
2.1.1. Actors of the System	31
2.2. Description of Every User	33
2.2.1. Join Call Room.....	33
2.2.2. Start Video Communication.....	34
2.2.3. Detect Hand Landmarks.....	35
2.2.4. Recognize ASL Sign.....	37
2.2.5. Generate Text From Sign	38
2.2.6. Send Text Messages	39
2.2.7. Convert Text To Speech	41
2.2.8. Leave Call Room	42
2.3. Domain Model	44
Chapter 3	45
3.1. Design Class Diagram	45
3.2. Entity Relationship Diagram.....	46
3.3. Sequence Diagram of Each SSD.....	47
3.3.1. Join Room Call.....	47
3.3.2. Start Video Communication.....	48
3.3.3. Detect Hand Landmarks.....	48
3.3.4. Recognize ASL Sign.....	49

3.3.5. Generate Text From Signs	49
3.4. System Architecture Diagram	50
3.4.1. Architecture Overview	50
3.5. Database Design Details	51
3.5.1 Current Database Usage.....	51
3.5. UI/UX Mockups or Wireframes	52
3.6.1. Home page wireframe	52
3.5.2. Call Room Page Wireframe	52
3.6.3. Call Header Section.....	52
3.5.3. Video Grid Section.....	53
3.5.4. Sign Player Section	53
3.5.5. Word Builder Section	53
3.5.6. Speech Input Section	54
Chapter 4	55
4.1. Code / Algorithm of Main Scenarios and Main Units.....	55
4.1.1. Project Structure	55
4.1.2. Join Call Room.....	55
4.1.3. Start Video Call	55
4.1.4. Hand Landmark Detection.....	56
4.1.5. ASL Sign Recognition	57
4.1.6. Build and Send Sentence.....	57
4.1.7. Speech to Text to Signs.....	58
4.1.8. WebSocket Message Broadcasting.....	59
4.1.9. Machine Learning Training Algorithm	59
4.2. Tools and Technologies Used	60
4.2.1. Frontend Technologies.....	60
4.2.2. Backend Technologies	60
4.2.3. Machine Learning Technologies.....	60
4.2.4. External Services	60
4.3. System Modules Description	61
4.3.1. Frontend Modules	61
4.3.2. Backend Modules	62

4.3.4. Machine Learning Modules	62
4.4. Deployment Details	63
4.4.1. Project Setup Steps	63
4.4.2. Deployment Architecture	64
4.4.3. Deployment Limitations	64
Chapter 5	65
5.1. Test Case	65
5.1.1. Test Case Table	65
5.1.2. Integration Testing	69
5.1.3. Final Testing Conclusion.....	70
Chapter 6	71
6.1. Home Page	71
6.2. Video Call Interface	72
6.3. Detect Hand Landmarks	73
6.3. Sign Recognition and message transfer.....	73
6.4. Message Send.....	74
6.5. Speech to Sign	75

List of Figures

Figure 1:Agile Sprint Schedule.....	30
Figure 2:Use Case model.....	32
Figure 3:Domain Model	44
Figure 4:Design Class Diagram.....	45
Figure 5:Entity Relationship Diagram	46
Figure 6:Seq Diagram Join Call Room.....	47
Figure 7:Start Video Communication	48
Figure 8: Detect Hand Landmarks	48
Figure 9:Recognize ASL.....	49
Figure 10: Generate Text From Sign	49
Figure 11:Architecture Overview.....	50
Figure 12: Current Database Usage details	51
Figure 13: Home page wireframe	52
Figure 14: Call Room Page Wireframe	52
Figure 15: Speech Input Section	54
Figure 16:Home page	71
Figure 17:Video Call Interface	72
Figure 18:Detect Hand Landmarks	73
Figure 19: Sign Detection	74
Figure 20:Message Send	74
Figure 21: Convert Speech into sign.....	75

List of Tables

Table 1:Join Call Room	21
Table 2: Access Camera	22
Table 3:Establish Video Call.....	22
Table 4:Connect To Signaling Server	23
Table 5:Detect Hand Marks	24
Table 6:Recognize ASL Signs	25
Table 7: Build Text from Recognized Signs	26
Table 8:Leave Room Call	27
Table 9:Agile Sprint Schedule – Sign Bridge Project	30
Table 10:Join Call Room.....	34
Table 11:Start Video Communication	35
Table 12:Detect Hand Landmarks	37
Table 13:Recognize ASL Sign	38
Table 14:Generate Text From Sign	39
Table 15:Send Text Messages	40
Table 16:Convert Text To Speech	42
Table 17:Leave Call Room.....	43
Table 18:Frontend Technologies	60
Table 19:Backend Technologies	60
Table 20:ML Technologies	60
Table 21:Join call with valid room ID	65
Table 22:Join call with empty room ID	66
Table 23:Camera permission granted	67
Table 24:Camera permission Rejected.....	67
Table 25:Connect two users in same room.....	68
Table 26:Third user joins room.....	69
Table 27:Hand shown to camera	69

ABSTRACT

Communication between sign language users and non-sign language users remains a major challenge in everyday life, education, and social interaction. Many hearing-impaired or speech-impaired individuals depend on sign language, while others rely on spoken language, creating a communication gap that limits effective interaction. To address this problem, this project presents SignBridge, a real-time web-based communication system that combines video calling with American Sign Language (ASL) recognition and bidirectional message conversion.

SignBridge is developed using a modern three-tier architecture. The frontend is built with React and Vite, providing an interactive user interface for room-based video communication. The backend is implemented using Django and Django Channels, which handle real-time WebSocket signaling for WebRTC peer-to-peer video calls. Sign recognition is performed in the browser using MediaPipe for hand landmark detection and a trained machine learning model exported as a JSON-based dense neural network. The system supports real-time ASL alphabet recognition, text generation, sentence normalization, text-to-speech, speech-to-text input, and visual sign playback for incoming spoken messages.

The system enables two-way communication between users. A sign language user can perform ASL signs in front of the camera, and the system converts recognized gestures into text that is sent to the other participant and spoken using text-to-speech. A non-sign language user can speak into the microphone, and the system converts the speech into text and displays supported ASL signs to the sign language user through a sign playback module. This makes SignBridge an assistive communication platform that supports both sign-to-text and speech-to-sign interaction.

The machine learning model was trained on hand landmark features extracted from an ASL dataset and achieved approximately 99% test accuracy on supported static alphabet signs. The final system demonstrates successful integration of video communication, real-time hand tracking, browser-based inference, and accessible messaging features. SignBridge provides a practical prototype for improving communication accessibility and can be extended in future work through dynamic gesture recognition, expanded sign vocabulary, user authentication, and cloud deployment.

Chapter 1

SOFTWARE REQUIREMENT SPECIFICATION

A Software Requirement Specification (SRS) is a formal document that describes the functional and non-functional requirements of a software system in a clear, structured, and detailed manner. It serves as a communication bridge between clients, developers, designers, and testers by defining what the system should do, how it should perform, and the constraints under which it will operate. The SRS document includes system features, user requirements, hardware and software requirements, performance expectations, security needs, and overall project scope. It acts as a blueprint for the development process, helping ensure that the final product meets user expectations while reducing misunderstandings, development errors, and project risks.

1.1. Introduction

Sign Bridge is a web-based real-time communication system designed to support communication between sign language users and non-sign language users. The system provides a video calling environment where users can join a call room and communicate through live video. Along with video communication, the system also includes sign language recognition, which detects hand gestures from the user's camera and converts recognized signs into readable text.

The project uses modern web and machine learning technologies to provide an assistive communication solution. The frontend of the system is developed using React, while the backend is developed using Django and Django Channels. WebRTC is used for real-time peer-to-peer video communication, and Web Sockets are used for signaling between users. For sign recognition, the system uses Media Pipe to detect hand landmarks and a trained machine learning model to classify ASL signs.

The main purpose of this project is to reduce the communication gap between people who use sign language and people who do not understand sign language. In many real-life situations, hearing-impaired or speech-impaired people face difficulty while communicating with others. Sign-Bridge helps by recognizing hand signs and converting them into text. The generated text can also be spoken using text-to-speech, making the communication more accessible.

This system is especially useful in educational, personal, and assistive communication environments. It provides a basic but effective platform where users can perform ASL alphabet signs during a video call and send the generated text message to the other participant. Although the current version focuses mainly on static ASL alphabet recognition, it provides a strong foundation for future improvements such as sentence-level translation, dynamic gesture recognition, and multi-language sign support.

1.2. Project Objectives

The objectives of the Sign Bridge project define the main purpose and expected outcomes of the system. These objectives explain what the project aims to achieve and how it will solve the communication problem between sign language users and non-sign language users. The objectives are divided into two categories: **general objective** and **specific objectives**.

1.2.1. General Objective

The general objective of this project is to develop a real-time web-based communication system that combines video calling with sign language recognition. The system aims to help hearing-impaired or speech-impaired users communicate more easily with people who may not understand sign language. Sign-Bridge provides a platform where users can join a video call, perform hand signs in front of the camera, and have those signs converted into readable text. This improves accessibility and makes communication more inclusive. The system is designed to support basic ASL alphabet recognition and assist users in forming words or simple sentences during live communication.

In simple words, the main goal of this project is to reduce the communication gap between sign language users and ordinary users through technology.

1.2.2. Specific Objectives

The specific objectives of the Sign-Bridge system are listed below:

1. **To develop a web-based communication platform**
The system should provide a simple web interface where users can access the application through a browser without installing a separate desktop application.
2. **To allow users to join a call room using a room ID**
Users should be able to enter a room ID and join a specific video call room. This makes the system simple and easy to use for two-person communication.
3. **To implement real-time video communication**
The system should support live video calling between two users. WebRTC is used to establish peer-to-peer video communication.
4. **To implement real-time signaling using WebSockets**
The backend should provide real-time signaling support so that users can exchange WebRTC connection data such as offer, answer, and ICE candidates.
5. **To detect hand landmarks from live camera feed**
The system should detect hand points from the user's camera using MediaPipe. These landmarks are used as input features for sign recognition.
6. **To recognize ASL alphabet signs**
The system should classify detected hand landmarks into supported ASL alphabet signs using a trained machine learning model.
7. **To convert recognized signs into text**
The system should allow recognized signs to form letters, words, and simple sentences.
8. **To provide text-to-speech support**
The system should convert generated or received text into speech so that the message can also be heard by the other user.
9. **To allow text message sharing during the call**
The system should allow users to send the generated sentence to the other participant in the video call.

10. To provide a user-friendly interface

The system should be simple, clear, and easy to operate for both technical and non-technical users.

11. To create a foundation for future improvement

The system should be designed in a modular way so future developers can add more signs, dynamic gesture recognition, sentence-level translation, or support for other sign languages.

1.3. Scope of the System

The scope of the Sign-Bridge system defines the boundaries, features, and limitations of the project. It explains what the system is designed to do and what is outside the current version of the project.

Sign-Bridge is developed as a real-time web-based communication platform that supports video calling and ASL sign recognition. The system allows two users to join the same call room using a room ID. After joining the room, both users can communicate through live video. Along with video communication, the system detects hand gestures from the local camera and recognizes supported ASL signs.

The main focus of the system is to assist communication between sign language users and non-sign language users. A user can perform ASL alphabet signs in front of the camera, and the system converts the recognized signs into text. The generated text can be sent to the other participant in the call. The system also supports text-to-speech, so the generated or received message can be spoken by the browser.

The current scope of the system includes:

1. Web-based access

The system runs in a web browser, so users do not need to install a separate desktop application.

2. Room-based video calling

Users can enter a room ID and join a specific video call room.

3. Two-user communication

The current version supports video communication between two users in one room.

4. Real-time video streaming

WebRTC is used to provide peer-to-peer video communication.

5. Real-time signaling

Django Channels and WebSockets are used to exchange signaling messages between users.

6. Hand landmark detection

MediaPipe is used to detect hand landmarks from the live camera feed.

7. ASL sign recognition

A trained machine learning model is used to classify supported ASL alphabet signs.

8. Text generation

Recognized signs are combined to form letters, words, and simple sentences.

9. Message sharing

Users can send generated text messages to the other participant during the call.

10. Text-to-speech support

The system can speak generated or received text messages using browser speech synthesis.

The current system is mainly focused on static ASL alphabet recognition. It does not provide full natural language translation of sign language. It also does not currently support complex dynamic gestures, multi-user conference calling, user accounts, chat history storage, or mobile application deployment.

Therefore, the scope of Sign-Bridge is limited to a prototype-level assistive communication system that demonstrates how video calling and sign recognition can be combined to improve communication accessibility. It can be expanded in the future by adding dynamic gesture recognition, more sign vocabulary, sentence-level interpretation, user authentication, cloud deployment, and support for more than two users.

1.4. Technical Feasibility

Technical feasibility explains whether the proposed system can be developed using available technology, tools, hardware, and software resources. The Sign-Bridge project is technically feasible because it uses modern, reliable, and widely supported technologies for web development, real-time communication, and machine learning.

The frontend of the system is developed using **React** with **Vite**. React helps in creating an interactive user interface, while Vite provides fast development and efficient build support. These tools are suitable for creating a browser-based application where users can join rooms, view video streams, see detected signs, and interact with text controls.

For real-time video communication, the system uses **WebRTC**. WebRTC is supported by modern web browsers and allows peer-to-peer audio/video communication without requiring a separate streaming server. This makes it suitable for live video calling between two users. In this project, WebRTC is used to connect two users in the same room and display their video streams.

The backend is developed using **Django** and **Django Channels**. Django provides a strong backend framework, while Django Channels adds WebSocket support. WebSockets are required for real-time signaling between two users. The signaling server exchanges connection information such as offer, answer, and ICE candidates so that WebRTC can establish the video call.

For sign recognition, the system uses **MediaPipe** for hand landmark detection. MediaPipe is capable of detecting hand points from live camera frames in real time. These hand landmarks are then passed to a trained machine learning model that classifies the sign. The model is exported into a browser-readable format, so prediction can be performed directly in the frontend. This reduces backend load and improves response time.

The machine learning part of the project is developed using **Python**, **TensorFlow/Keras**, **OpenCV**, **MediaPipe**, and **scikit-learn**. These tools are commonly used for computer vision and machine learning projects. The ML pipeline supports data collection, landmark extraction, model training, evaluation, and model export.

The hardware requirements are also reasonable. A standard laptop or desktop computer with a webcam is enough to run and test the system. Since the model is lightweight and runs on hand landmark features instead of full image frames, it does not require high-end GPU hardware for normal use.

Overall, the project is technically feasible because all required technologies are available, open-source, and compatible with each other. The selected tools are suitable for implementing real-time video communication, hand tracking, sign recognition, and browser-based interaction.

1.5. Economic Feasibility

Economic feasibility explains whether the project can be developed and operated within reasonable cost. The Sign-Bridge system is economically feasible because it is built mainly using open-source technologies and does not require expensive hardware or paid software licenses.

The main development tools used in the project, such as **React**, **Vite**, **Django**, **Django Channels**, **Python**, **TensorFlow/Keras**, **OpenCV**, **MediaPipe**, and **scikit-learn**, are freely available. These technologies can be used without purchasing commercial licenses. This reduces the overall development cost of the project.

The system can be developed and tested on a normal laptop or desktop computer with a webcam. No special hardware device is required for basic operation. Since the sign recognition model works on hand landmark features instead of processing large video frames on the server, the system does not need a powerful backend server for prediction during normal browser-based use.

The backend server is mainly used for WebSocket signaling and optional API support. Therefore, it can run on a low-cost local machine or a basic cloud server during demonstration or deployment. Video communication is handled through WebRTC peer-to-peer connection, which reduces server bandwidth and hosting cost because video streams do not pass through the backend server.

The project also has low maintenance cost because the system is modular. The frontend, backend, and machine learning components are separated, so each part can be updated independently. Future improvements such as adding more signs, improving the model, or deploying the backend can be done gradually without rebuilding the whole system.

Overall, Sign-Bridge is economically feasible because it uses free tools, requires only standard hardware, reduces server load through browser-based inference and WebRTC, and can be maintained with limited resources.

1.6. Operational Feasibility

Operational feasibility explains whether the system can be used effectively by its intended users in a real working environment. The Sign-Bridge system is operationally feasible because it provides a simple browser-based interface and does not require complex user training.

The system is designed for users who need assistance in communication, especially hearing-impaired or speech-impaired users and people who do not understand sign language. A user can open the application in a modern web browser, enter a room ID, and join a video call.

The system then uses the camera to detect hand gestures and convert recognized ASL signs into text.

The operation of the system is simple. The user only needs a device with a camera, browser support, and an internet or local network connection. After joining a call, the user can perform signs in front of the camera. The system detects the hand, predicts the sign, builds text, and allows the user to send the message to the other participant. The text-to-speech feature also helps by speaking the generated or received message.

The system is also easy to operate because it includes basic controls such as joining a room, turning the camera on or off, removing the last letter, adding space, clearing text, speaking preview, sending sentence, and leaving the call. These controls make the system manageable for both technical and non-technical users.

In practical use, the system requires proper lighting, a clear camera view, and correct hand positioning for better sign recognition accuracy. Since the system depends on camera input and hand landmark detection, poor lighting or unclear gestures may reduce recognition performance. However, these operational limitations can be handled by guiding the user to use the system in a suitable environment.

1.7. Stakeholders

Stakeholders are the individuals, groups, or organizations that are directly or indirectly affected by the Sign-Bridge system. They may use the system, benefit from it, evaluate it, maintain it, or support its development.

The main stakeholders of the Sign-Bridge project are described below.

1.7.1. Hearing-Impaired or Speech-Impaired Users

Hearing-impaired or speech-impaired users are the primary beneficiaries of the system. They can use Sign-Bridge to communicate through hand signs during a video call. The system recognizes supported ASL signs and converts them into text, which helps these users express their message to people who may not understand sign language.

1.7.2. Non-Sign Language Users

Non-sign language users are also important stakeholders. These users may not be able to understand ASL signs directly. Sign-Bridge helps them by displaying the recognized signs as text and by speaking the message through text-to-speech. This makes communication easier between sign language users and ordinary users.

1.7.3. Students and Researchers

Students and researchers can use this project as a learning and research-based system in the areas of computer vision, machine learning, assistive technology, and real-time communication. The project demonstrates how technologies like WebRTC, MediaPipe, Django Channels, and machine learning can be combined in one system.

1.7.4. Project Supervisor and Evaluation Committee

The project supervisor and evaluation committee are stakeholders because they guide, review, and evaluate the project. They are interested in the correctness, feasibility, implementation quality, documentation, and final outcome of the system.

1.7.5. Educational Institutions

Educational institutions may benefit from this project as an academic and assistive technology solution. The project can be used for demonstration, research, and future extension by students working on similar areas.

1.7.6. Developers and Maintainers

Developers and maintainers are responsible for improving, fixing, deploying, and extending the system. They may add more signs, improve the model accuracy, enhance the user interface, or deploy the system on a production server.

1.8. User of the System

Users of the system are the people who directly interact with Sign-Bridge. Since the system is designed for real-time video communication and sign recognition, the main users are those who either use sign language or need to understand sign-based communication.

The users of the Sign-Bridge system are described below.

1.8.1. Sign Language User

The sign language user is the main user of the system. This user performs hand signs in front of the camera during a video call. The system detects the hand landmarks, recognizes the supported ASL sign, and converts it into text.

This user can build words and sentences from recognized signs. After forming a message, the user can send it to the other participant in the call. This helps the sign language user communicate with someone who may not understand sign language.

1.8.2. Non-Sign Language User

The non-sign language user is the person who communicates with the sign language user but may not understand ASL signs. This user joins the same video call room and receives the generated text message.

The system helps this user by showing the recognized message in text form. The text-to-speech feature can also speak the message, making it easier to understand. In this way, the system supports communication between both users.

1.8.3. General Communication User

A general communication user may use the system simply for video calling and text-based communication. This user can join a room, view the other participant's video, send generated text, and use available controls such as camera toggle, speak preview, and leave call.

1.8.4. Developer or Administrator

The developer or administrator is a technical user who is responsible for running, maintaining, testing, and improving the system. This user may configure the backend server, update the frontend, improve the machine learning model, or fix system issues.

The developer may also add new signs, improve recognition accuracy, deploy the system, or manage the project files. Although the current system does not include a separate admin panel for end-user management, Django provides backend administrative support if required in future versions.

1.9. Functional Requirements

Functional requirements describe the main services and functions that the Sign-Bridge system must provide to its users. These requirements explain what the system should do during normal operation.

In Sign-Bridge, the main functional requirements are related to joining a call room, starting video communication, detecting hand signs, generating text, sending messages, and managing the call session.

Description of Each Requirement in Requirement Shell

1.9.1. Join Call Room

Requirement Name: Join Call Room

Requirement #:	Req01	Requirement Type:	Functional
Description:	The system shall allow the user to enter a room ID and join a video call room. The room ID is used to connect two users in the same communication session.		
Rationale:	Users must be able to join a specific room to begin communication.		
Originator:	Sign Language User, Non-Sign Language User		
Input:	Room ID entered by the user.		
Processing:	The system checks whether the room ID is not empty and then redirects the user to the call room page.		

Output:	The user enters the selected call room.		
Fit Criterion:	User is successfully redirected to the call room when a valid room ID is entered.		
Customer Satisfaction:	5	Customer Dissatisfaction:	5
Priority:	High	Conflicts:	Nil
Supporting Materials:	NA		
History:			

Table 1:Join Call Room

1.9.2. Access Camera

Requirement Name: Access Camera

Requirement #:	Req02	Requirement Type:	Functional
Description:	The system shall request permission to access the user's camera. Camera access is required for video calling and hand sign detection.		
Rationale:	Camera access is essential for both video communication and sign recognition.		
Originator:	Sign Language User, Non-Sign Language User		
Input:	Browser camera permission from the user.		
Processing:	The system requests camera access through the browser. If permission is granted, the camera stream is started.		
Output:	The user's live video appears on the call screen.		
Fit Criterion:	Live video stream is displayed on the call screen after permission is granted.		
Customer Satisfaction:	5	Customer Dissatisfaction:	5

Priority:	High	Conflicts:	Nil
Supporting Materials:	NA		
History:			

Table 2: Access Camera

1.9.3. Establish Video Call

Requirement Name: Establish Video Call

Requirement #:	Req03	Requirement Type:	Functional
Description:	The system shall establish a real-time video call between two users who join the same room.		
Rationale:	Real-time video communication is the core feature of the system.		
Originator:	Sign Language User, Non-Sign Language User		
Input:	Two users joining the same room ID.		
Processing:	The system uses WebRTC to create a peer-to-peer video connection between both users.		
Output:	Both users can see each other's video streams.		
Fit Criterion:	Both users see live video streams of each other in the same room.		
Customer Satisfaction:	5	Customer Dissatisfaction:	5
Priority:	High	Conflicts:	Nil
Supporting Materials:	NA		
History:			

Table 3: Establish Video Call

1.9.4. Connect To Signaling Server

Requirement Name: Connect To Signaling Server

Requirement #:	Req04	Requirement Type:	Functional
Description:	The system shall connect to the backend signaling server using WebSockets. The signaling server helps both users exchange WebRTC connection information.		
Rationale:	WebSocket signaling is required to initiate and establish the WebRTC video call.		
Originator:	System (Backend)		
Input:	Room ID and WebSocket connection request.		
Processing:	The backend assigns user roles such as caller and callee, then exchanges offer, answer, and ICE candidate messages.		
Output:	A successful signaling connection is established between users.		
Fit Criterion:	WebRTC connection is successfully established through signaling exchange.		
Customer Satisfaction:	5	Customer Dissatisfaction:	5
Priority:	High	Conflicts:	Nil
Supporting Materials:	NA		
History:			

Table 4: Connect To Signaling Server

1.9.5. Detect Hand Marks

Requirement Name: Detect Hand Marks

Requirement #:	Req05	Requirement Type:	Functional
-----------------------	-------	--------------------------	------------

Description:	The system shall detect hand landmarks from the user's live camera feed. These landmarks represent important points of the hand such as fingers and wrist.		
Rationale:	Hand landmark detection is required as input for ASL sign classification.		
Originator:	Sign Language User		
Input:	Live video frames from the camera.		
Processing:	MediaPipe detects the hand and extracts landmark points from the video frame.		
Output:	Hand landmark coordinates are generated for sign recognition.		
Fit Criterion:	Hand landmark coordinates are extracted and available for model input in real time.		
Customer Satisfaction:	5	Customer Dissatisfaction:	5
Priority:	High	Conflicts:	Nil
Supporting Materials:	NA		
History:			

Table 5: Detect Hand Marks

1.9.6. Recognize ASL Signs

Requirement Name: Recognize ASL Signs

Requirement #:	Req06	Requirement Type:	Functional
Description:	The system shall recognize supported ASL signs using a trained machine learning model.		
Rationale:	Sign recognition is the main functionality that enables communication for sign language users.		

Originator:	Sign Language User		
Input:	Hand landmark features.		
Processing:	The system passes the extracted landmark features to the trained model. The model predicts the most likely ASL sign with a confidence value.		
Output:	Predicted ASL sign and confidence score.		
Fit Criterion:	A supported ASL sign is correctly identified and displayed with confidence score.		
Customer Satisfaction:	5	Customer Dissatisfaction:	5
Priority:	High	Conflicts:	Nil
Supporting Materials:	NA		
History:			

Table 6:Recognize ASL Signs

1.9.7. Build Text From Recognized Signs

Requirement Name: Build Text From Recognized Signs

Requirement #:	Req07	Requirement Type:	Functional
Description:	The system shall build words and sentences from recognized ASL signs.		
Rationale:	Text building allows the sign user to form complete messages to send to the other participant.		
Originator:	Sign Language User		
Input:	Recognized letters or control signs.		

Processing:	The system adds recognized letters to the current word. It can also add space, delete letters, and clear text based on user action or recognized sign.		
Output:	Generated word or sentence.		
Fit Criterion:	Recognized signs are correctly combined into words and sentences in the text builder.		
Customer Satisfaction:	5	Customer Dissatisfaction:	5
Priority:	High	Conflicts:	Nil
Supporting Materials:	NA		
History:			

Table 7: Build Text from Recognized Signs

1.9.8. Leave Room Call

Requirement Name: Leave Room Call

Requirement #:	Req08	Requirement Type:	Functional
Description:	The system shall allow the user to leave the current video call room.		
Rationale:	Users must be able to cleanly end the call and return to the home page.		
Originator:	Sign Language User, Non-Sign Language User		
Input:	Leave call button click.		
Processing:	The system closes the connection, stops the media stream, and redirects the user to the home page.		
Output:	The user exits the call room.		
Fit Criterion:	User is redirected to home page and all media streams are stopped after leaving.		

Customer Satisfaction:	5	Customer Dissatisfaction:	5
Priority:	High	Conflicts:	Nil
Supporting Materials:	NA		
History:			

Table 8: Leave Room Call

1.9. Non Functional Requirements

Nonfunctional requirements describe the quality attributes of the system. These requirements explain how the system should perform rather than what specific function it should perform. For the Sign-Bridge system, nonfunctional requirements are important because the system depends on real-time communication, camera input, machine learning prediction, and user interaction.

1.10.1. Performance

The system should perform sign detection and video communication in real time. The user's hand signs should be detected quickly from the camera feed, and the predicted result should be displayed with minimum delay. The video call should also run smoothly so that both users can communicate without major interruption. Since the system performs sign prediction in the browser using hand landmarks, it should avoid unnecessary backend processing. This improves response time and reduces network delay. The system should work efficiently on standard laptops or desktops with webcam support.

1.10.2. Usability

The system should be easy to use for both technical and non-technical users. The interface should be simple, clear, and understandable. Users should be able to enter a room ID, join a call, see video streams, view detected signs, build text, send messages, and leave the call without confusion. Buttons and controls should be clearly labeled, such as "Join Call," "Turn Camera Off," "Send Sentence," "Clear," and "Leave Call." This helps users operate the system without requiring detailed technical knowledge.

1.10.3. Reliability

The system should maintain stable communication during a call. The WebSocket connection should support signaling between both users, and the WebRTC connection should allow continuous video streaming. If one user leaves the call, the other user should be informed. The system should also handle common situations such as no hand detected, unknown signs, camera permission denial, or connection issues. These messages help users understand the current system state.

1.10.4. Accessibility

The system should improve communication accessibility for users who rely on sign language. It should convert recognized signs into text and provide text-to-speech support. This allows messages to be both read and heard by the receiving user. The system should also provide visual feedback such as detected sign, current word, sentence, and call status. These features make the application more helpful and understandable during communication.

1.10.5. Compatibility

The system should work on modern web browsers that support camera access, WebRTC, WebSockets, and speech synthesis. Since the project is browser-based, users should not need to install a separate desktop application. The system should run on common operating systems such as Windows, Linux, or macOS, as long as the browser supports the required web APIs.

1.10.6. Security

The system should request camera permission through the browser before accessing the user's video. It should not access the camera without user approval. The system should only use the camera for video communication and sign detection. For future deployment, security can be improved by using HTTPS, secure WebSocket connections, environment variables for secret keys, and proper CORS configuration.

1.10.7. Maintainability

The system should be easy to maintain and improve. The project is divided into separate parts: frontend, backend, and machine learning pipeline. This modular structure helps developers update one part of the system without affecting the entire project. For example, the frontend can be improved without changing the ML training scripts, and the ML model can be retrained or replaced without rewriting the full backend.

1.10.8. Scalability

The current version of the system supports two users per room. In future versions, the system should be extendable to support more users, more signs, better model accuracy, and production-level deployment. The backend currently handles signaling for rooms, but future scalability can be improved by using a production channel layer such as Redis and deploying the system on a cloud server.

1.10.9. Accuracy

The sign recognition model should provide accurate predictions for supported ASL signs. Recognition accuracy depends on model training, camera quality, lighting, hand position, and gesture clarity. The system should provide confidence-based prediction so that weak or uncertain signs are not accepted too quickly. This helps reduce incorrect text generation.

1.11. Schedule of Project

The Sign-Bridge project follows the Agile development methodology. Agile allows the project to be developed in short iterative cycles called Sprints. Each sprint focuses on a specific set of tasks and delivers a working increment of the system. This approach helped the team manage requirements, integrate components gradually, and test each part before moving to the next. The project was divided into 10 sprints covering requirement analysis, system design, backend and frontend development, machine learning integration, testing, and final documentation.

Table: Agile Sprint Schedule – Sign Bridge Project

Sprint	Sprint Goal	Key Tasks	Deliverable	Duration
Sprint 1	Requirement Analysis & Project Setup	Define functional/non-functional requirements; identify stakeholders and actors; select technologies (React, Django, WebRTC, MediaPipe, TensorFlow)	SRS Document, Technology Stack	2 Weeks
Sprint 2	System Architecture & Design	Design system architecture; create use case model, domain model, class diagram, ERD; plan database structure; design UI/UX wireframes	Architecture Diagram, UML Diagrams, Wireframes	2 Weeks
Sprint 3	Backend Development	Develop Django backend; implement Django Channels WebSocket consumer; set up room management and caller/callee role assignment; exchange WebRTC signaling messages	Working Signaling Server, WebSocket API	2 Weeks
Sprint 4	Frontend Development	Develop React frontend; create Home Page (room entry) and Call Room Page; implement camera access, video display, and call controls (toggle camera, leave call)	Working Frontend UI with Room Join & Video	2 Weeks
Sprint 5	WebRTC Video Integration	Integrate WebRTC peer-to-peer video calling with backend signaling; test offer/answer/ICE candidate exchange; verify two-user video communication	Live Video Call between Two Users	2 Weeks
Sprint 6	Machine Learning – Data & Training	Collect hand landmark data for ASL alphabet signs; extract MediaPipe landmarks; train classification model using TensorFlow/Keras; evaluate accuracy	Trained ASL Model (TFLite / TF.js format)	2 Weeks

Sprint 7	Sign Recognition Integration	Integrate MediaPipe hand landmark detection into frontend; load trained ML model in browser; implement real-time sign prediction and confidence filtering	Live Sign Recognition in Call Room	2 Weeks
Sprint 8	Text Builder & Message Sharing	Implement letter/word builder from recognized signs; add space, delete, and clear controls; send generated sentence via WebSocket to other participant; display received message	Message Send/Receive Feature	1 Week
Sprint 9	Text-to-Speech & Final Features	Implement browser text-to-speech for received messages; add speech preview feature; finalize all call controls; fix bugs from integration testing	Complete Working System with TTS	1 Week
Sprint 10	Testing & Documentation	Perform unit, integration, system, and UAT testing; log and resolve defects; prepare SRS, analysis, design, implementation and testing chapters; submit final documentation	Test Report, Final Documentation	2 Weeks

Table 9: Agile Sprint Schedule – Sign Bridge Project

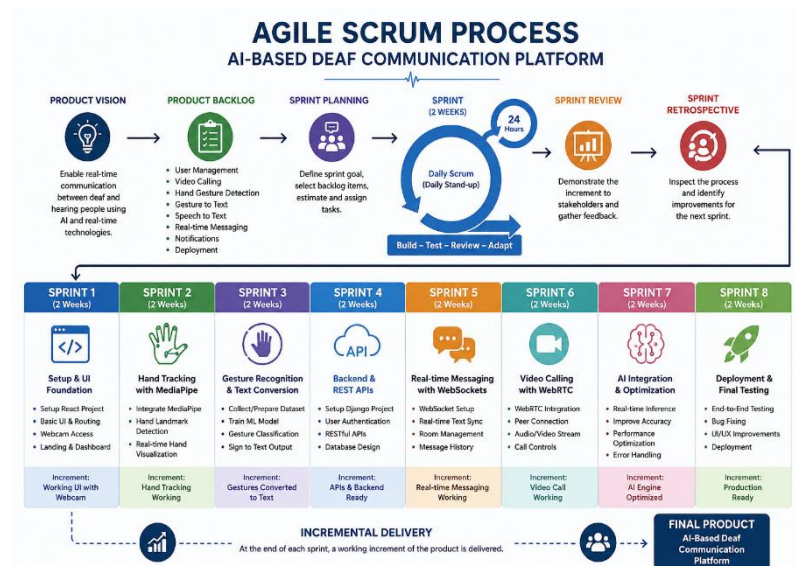


Figure 1: Agile Sprint Schedule

Chapter 2

ANALYSIS

The analysis phase is one of the most important stages in software development, as it focuses on understanding the problem, identifying user needs, and determining the system requirements. During this phase, the existing system is carefully studied to identify its limitations and challenges, while the proposed system is analyzed to provide improved solutions. The analysis process includes gathering requirements from users, examining system feasibility, defining functional and non-functional requirements, and evaluating technical, operational, and economic aspects of the project. This phase helps developers gain a clear understanding of the project objectives and ensures that the software system is designed according to user expectations and organizational goals.

2.1. Use Case Model

A use case model represents the interaction between users and the system. It shows what services the system provides and which users interact with those services. In the Sign-Bridge system, the main users are the sign language user, the non-sign language user, and the developer or administrator.

The system mainly provides video calling, sign recognition, text generation, message sharing, and speech output features.

2.1.1. Actors of the System

The main actors of the Sign-Bridge system are:

2.1.1.1. Sign Language User

The sign language user performs ASL signs in front of the camera. The system detects the hand signs, recognizes them, converts them into text, and allows the user to send the generated message to the other participant.

2.1.1.2. Non-Sign Language User

The non-sign language user joins the call and receives the generated text message. This user may not understand sign language, so the system helps by showing the translated text and speaking it through text-to-speech.

2.1.1.3. Developer

The developer or administrator is responsible for maintaining the system, running the backend server, improving the ML model, and managing future updates. This actor is not a normal end user but is important for system maintenance.

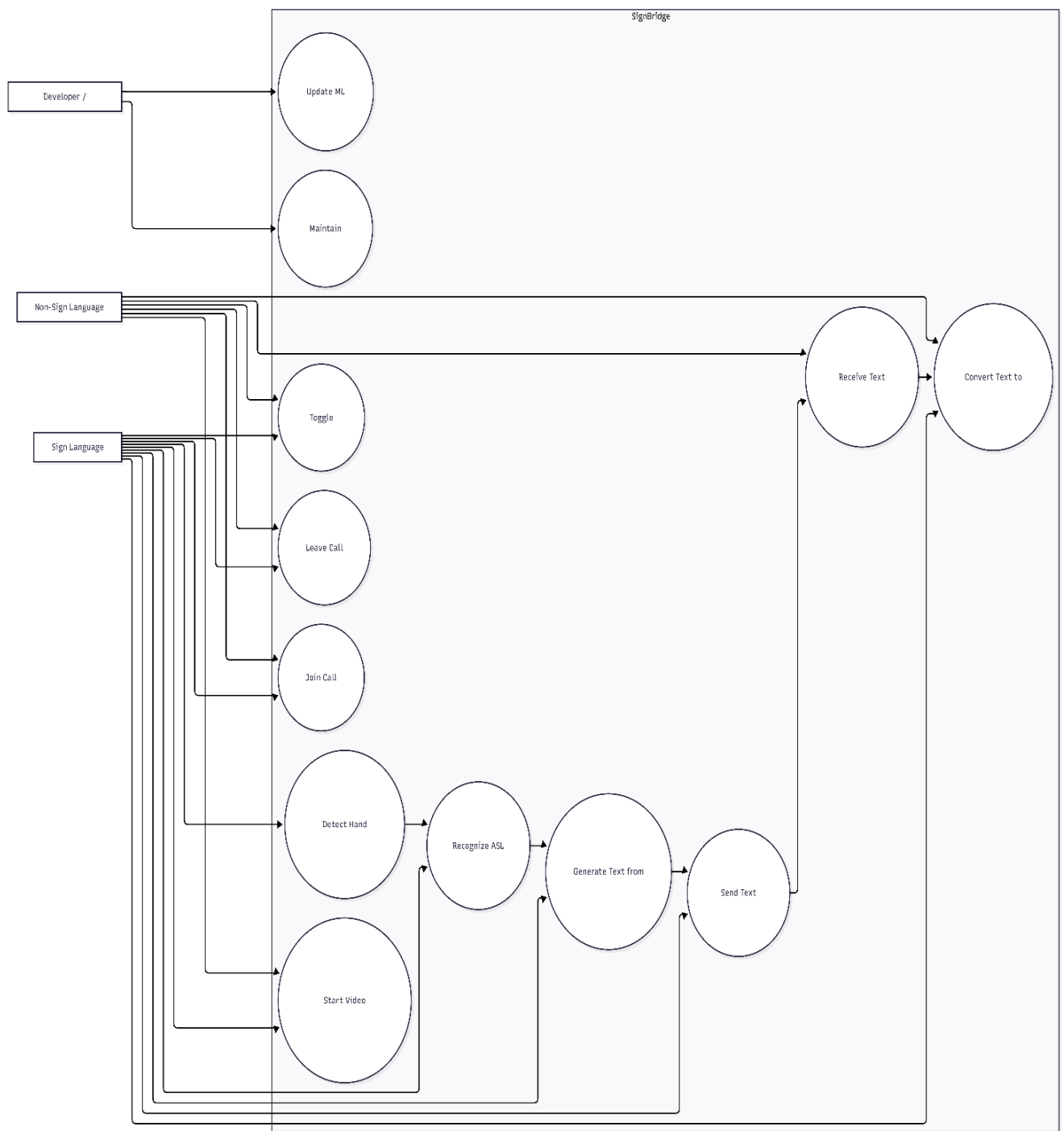


Figure 2: Use Case model

2.2. Description of Every User

2.2.1. Join Call Room

Use Case Name: Join Call Room (Ref. Req01)

UC #	UC-01	Ref:	Req01
UC Name	Join Call Room		
Level	Detailed		
Description	A user enters a room ID on the home page and is redirected to the call room to begin communication.		
Actor(s)	Sign Language User, Non-Sign Language User		
Stakeholders	Sign Language User, Non-Sign Language User, Project Supervisor		
Preconditions	The system is running. The user has a browser with camera support.		
Success Guarantee	The user is redirected to the call room page with the entered room ID.		
Main Success Scenario			
Action		Response	
1. User opens the Sign-Bridge home page.			
2.		System displays the home page with a room ID input field.	
3. User enters a valid room ID and clicks Join.			
4.		System validates the room ID is not empty.	
5.		System redirects the user to the call room page with the entered room ID.	
Extensions			

4a	Room ID is empty: System displays a validation message asking the user to enter a room ID.
Special Requirements	The room ID can be any non-empty string. No authentication is required.
Frequency of Occurrence	Each time a user wants to start or join a video call.
Miscellaneous	Two users must enter the same room ID to be connected.

Table 10:Join Call Room

2.2.2. Start Video Communication

Use Case Name: Start Video Communication (Ref. Req02)

UC #	UC-02	Ref:	Req02
UC Name	Start Video Communication		
Level	Detailed		
Description	After joining the call room, the system requests camera access and establishes a WebRTC video call between two users.		
Actor(s)	Sign Language User, Non-Sign Language User		
Stakeholders	Sign Language User, Non-Sign Language User		
Preconditions	Both users have joined the same room ID. Browser supports WebRTC and camera.		
Success Guarantee	Both users can see each other's live video stream in the call room.		
Main Success Scenario			
Action		Response	
1. User joins the call room.			
2.		System connects to the backend via WebSocket signaling.	

3.	System requests camera permission from the browser.
4. User grants camera permission.	
5.	System starts the local video stream and assigns caller/callee roles.
6.	System exchanges WebRTC offer, answer, and ICE candidates via signaling.
7.	System establishes peer-to-peer WebRTC connection between both users.
8.	Both users see each other's live video.
Extensions	
3a	User denies camera permission: System shows an error. Video call cannot proceed without camera access.
Special Requirements	Requires a stable internet or local network connection for WebRTC peer connection.
Frequency of Occurrence	Once per call session per user.
Miscellaneous	The backend only handles signaling; video streams are transmitted peer-to-peer.

Table 11:Start Video Communication

2.2.3. Detect Hand Landmarks

Use Case Name: Detect Hand Landmarks (Ref. Req05)

UC #	UC-03	Ref:	Req05
UC Name	Detect Hand Landmarks		
Level	Detailed		

Description	During the call, the system continuously detects hand landmarks from the user's live camera feed using MediaPipe.		
Actor(s)	Sign Language User		
Stakeholders	Sign Language User		
Preconditions	Camera is active. The user's hand is visible in the camera frame.		
Success Guarantee	Hand landmark coordinates are extracted from the camera frame for sign recognition.		
Main Success Scenario			
Action		Response	
1.		System captures live video frames from the camera.	
2. Sign Language User performs a hand sign in front of the camera.			
3.		System passes the frame to MediaPipe hand detection.	
4.		MediaPipe detects the hand and extracts 21 landmark coordinates.	
5.		System sends landmark features to the ASL recognition model.	
Extensions			
4a	No hand is detected in the frame: System skips prediction for that frame. Detection resumes on the next frame.		
Special Requirements	Good lighting and clear hand positioning improve detection accuracy.		
Frequency of Occurrence	Continuously while the call room is active and camera is on.		

Miscellaneous	Landmark extraction runs in the browser to reduce server load.
----------------------	--

Table 12: Detect Hand Landmarks

2.2.4. Recognize ASL Sign

Use Case Name: Recognize ASL Sign (Ref. Req06)

UC #	UC-04	Ref:	Req06
UC Name	Recognize ASL Sign		
Level	Detailed		
Description	The system classifies detected hand landmark features using a trained machine learning model to identify the ASL sign.		
Actor(s)	Sign Language User		
Stakeholders	Sign Language User		
Preconditions	Hand landmarks have been extracted successfully.		
Success Guarantee	A predicted ASL sign label and confidence score are displayed.		
Main Success Scenario			
Action		Response	
1.		System receives hand landmark coordinates from MediaPipe.	
2.		System passes landmark features to the trained TensorFlow.js model.	
3.		Model predicts the ASL sign with a confidence score.	
4.		System checks if the confidence score meets the minimum threshold.	
5.		System displays the predicted sign and confidence on screen.	

Extensions	
4a	Confidence score is below threshold: System does not add the letter. Prediction is ignored.
Special Requirements	Only supported ASL alphabet signs are recognized. The model was trained on MediaPipe landmark features.
Frequency of Occurrence	Continuously during active sign detection.
Miscellaneous	Model runs entirely in the browser using TensorFlow.js.

Table 13: Recognize ASL Sign

2.2.5. Generate Text From Sign

Use Case Name: Generate Text From Sign (Ref. Req07)

UC #	UC-05	Ref:	Req07
UC Name	Generate Text From Sign		
Level	Detailed		
Description	The system builds words and sentences from the recognized ASL signs using a letter-by-letter word builder.		
Actor(s)	Sign Language User		
Stakeholders	Sign Language User, Non-Sign Language User		
Preconditions	An ASL sign has been recognized with sufficient confidence.		
Success Guarantee	Recognized letters are accumulated into the current word and sentence.		
Main Success Scenario			
Action		Response	
1.		System receives a recognized ASL letter.	

2.	System appends the letter to the current word in the text builder.
3. Sign Language User performs the SPACE sign or clicks Add Space.	
4.	System adds the current word to the sentence and clears the word buffer.
5. Sign Language User reviews the generated sentence.	
Extensions	
2a	User clicks Delete Last Letter: System removes the last letter from the current word.
2b	User clicks Clear All: System clears the entire sentence and word buffer.
Special Requirements	Delete, space, and clear controls are available both as recognized signs and UI buttons.
Frequency of Occurrence	Continuously during active sign communication.
Miscellaneous	Each letter is only added after the confidence threshold is met.

Table 14:Generate Text From Sign

2.2.6. Send Text Messages

Use Case Name: Send Text Messages (Ref. Req07)

UC #	UC-06	Ref:	Req07
UC Name	Send Text Messages		
Level	Detailed		
Description	The sign language user sends the generated sentence as a text message to the other participant via WebSocket.		

Actor(s)	Sign Language User	
Stakeholders	Sign Language User, Non-Sign Language User	
Preconditions	A sentence has been generated. Both users are connected in the same room.	
Success Guarantee	The generated sentence is sent and displayed to the other participant.	
Main Success Scenario		
Action	Response	
1. Sign Language User clicks the Send Sentence button.		
2.	System sends the sentence to the backend via WebSocket.	
3.	Backend broadcasts the message to the other user in the same room.	
4.	The other user's screen displays the received message.	
5.	System clears the sentence buffer after sending.	
Extensions		
1a	Sentence is empty: System does not send the message.	
Special Requirements	Messages are real-time only and not stored in the database.	
Frequency of Occurrence	Whenever the sign language user wants to send a completed sentence.	
Miscellaneous	Messages are transmitted through the Django Channels WebSocket consumer.	

Table 15: Send Text Messages

2.2.7. Convert Text To Speech

Use Case Name: Convert Text To Speech (Ref. Req07)

UC #	UC-07	Ref:	Req07
UC Name	Convert Text To Speech		
Level	Detailed		
Description	The system converts received or generated text to speech using the browser's speech synthesis API.		
Actor(s)	Non-Sign Language User, Sign Language User		
Stakeholders	Non-Sign Language User, Sign Language User		
Preconditions	A text message has been received or generated.		
Success Guarantee	The text is spoken aloud through the browser speech synthesis.		
Main Success Scenario			
Action		Response	
1. User clicks Speak Preview or the system receives a new message.			
2.		System calls the browser SpeechSynthesis API with the message text.	
3.		Browser reads the message aloud in the default language voice.	
Extensions			
2a	Browser does not support SpeechSynthesis: System silently skips speech output.		
Special Requirements	Uses the Web Speech API built into modern browsers. No external service required.		

Frequency of Occurrence	Whenever a message is received or the user previews their sentence.
Miscellaneous	Speech is local to each user's browser.

Table 16: Convert Text To Speech

2.2.8. Leave Call Room

Use Case Name: Leave Call Room (Ref. Req08)

UC #	UC-08	Ref:	Req08
UC Name	Leave Call Room		
Level	Detailed		
Description	The user clicks the Leave Call button to end the video call, stop all media streams, and return to the home page.		
Actor(s)	Sign Language User, Non-Sign Language User		
Stakeholders	Sign Language User, Non-Sign Language User		
Preconditions	The user is currently in an active call room.		
Success Guarantee	The user exits the call room. WebRTC connection and camera stream are stopped. User is redirected to home page.		
Main Success Scenario			
Action		Response	
1. User clicks the Leave Call button.			
2.		System stops the local media stream (camera/microphone).	
3.		System closes the WebRTC peer connection.	
4.		System sends a disconnect message to the signaling server.	
5.		System redirects the user to the home page.	

Extensions	
1a	User closes the browser tab directly: Media streams are released by the browser. Connection is terminated.
Special Requirements	None.
Frequency of Occurrence	Once per call session when the user decides to end the call.
Miscellaneous	Other participant is notified and their screen shows the call has ended.

Table 17: Leave Call Room

2.3. Domain Model

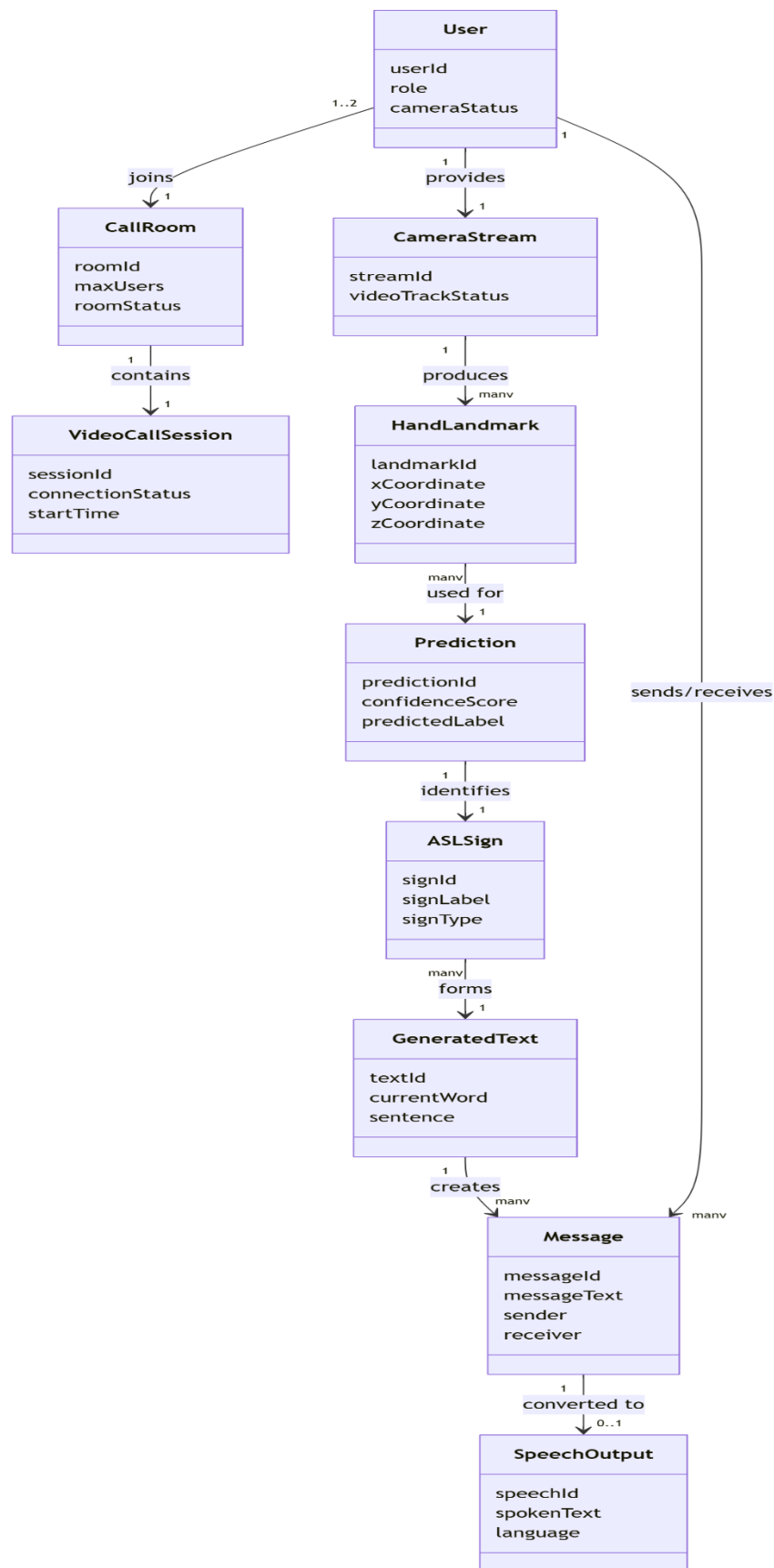


Figure 3:Domain Model

Chapter 3

PROJECT DESIGN

This chapter describes the design of the SignBridge system. The design phase explains how the system will be structured and how different components will work together to provide the required functionality. It includes diagrams and design-level details such as class diagram, entity relationship diagram, sequence diagrams, system architecture, database design, and UI/UX mockups. The purpose of this chapter is to convert the analysis from Chapter 2 into a practical system design. In this chapter, we will describe the frontend design, backend design, communication flow, machine learning integration, and user interface structure of the SignBridge project.

3.1. Design Class Diagram

A design class diagram represents the major software classes, components, and their relationships. It shows how the system is organized at the implementation level. Unlike the

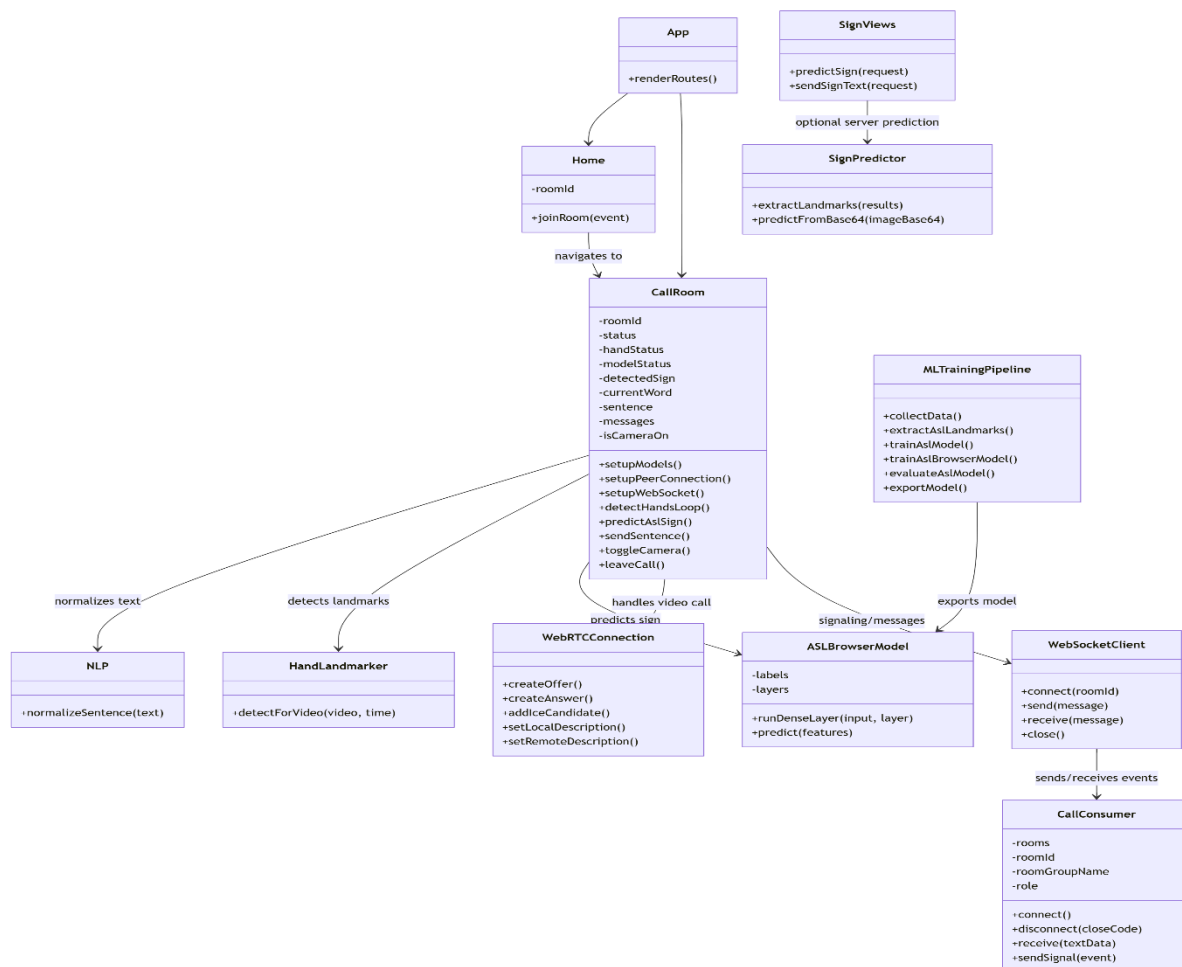


Figure 4: Design Class Diagram

domain model, which only shows real-world concepts, the design class diagram includes system classes and modules that are closer to the actual implementation. In the SignBridge system, the frontend is developed using React components and JavaScript functions, while the backend is developed using Django and Django Channels. The machine learning model is used for ASL sign prediction. Therefore, the design class diagram includes frontend components, backend services, WebSocket consumer, ML model, and supporting classes.

3.2. Entity Relationship Diagram

An Entity Relationship Diagram represents the data entities of a system and the relationships between them. It is normally used to design the database structure of a project.

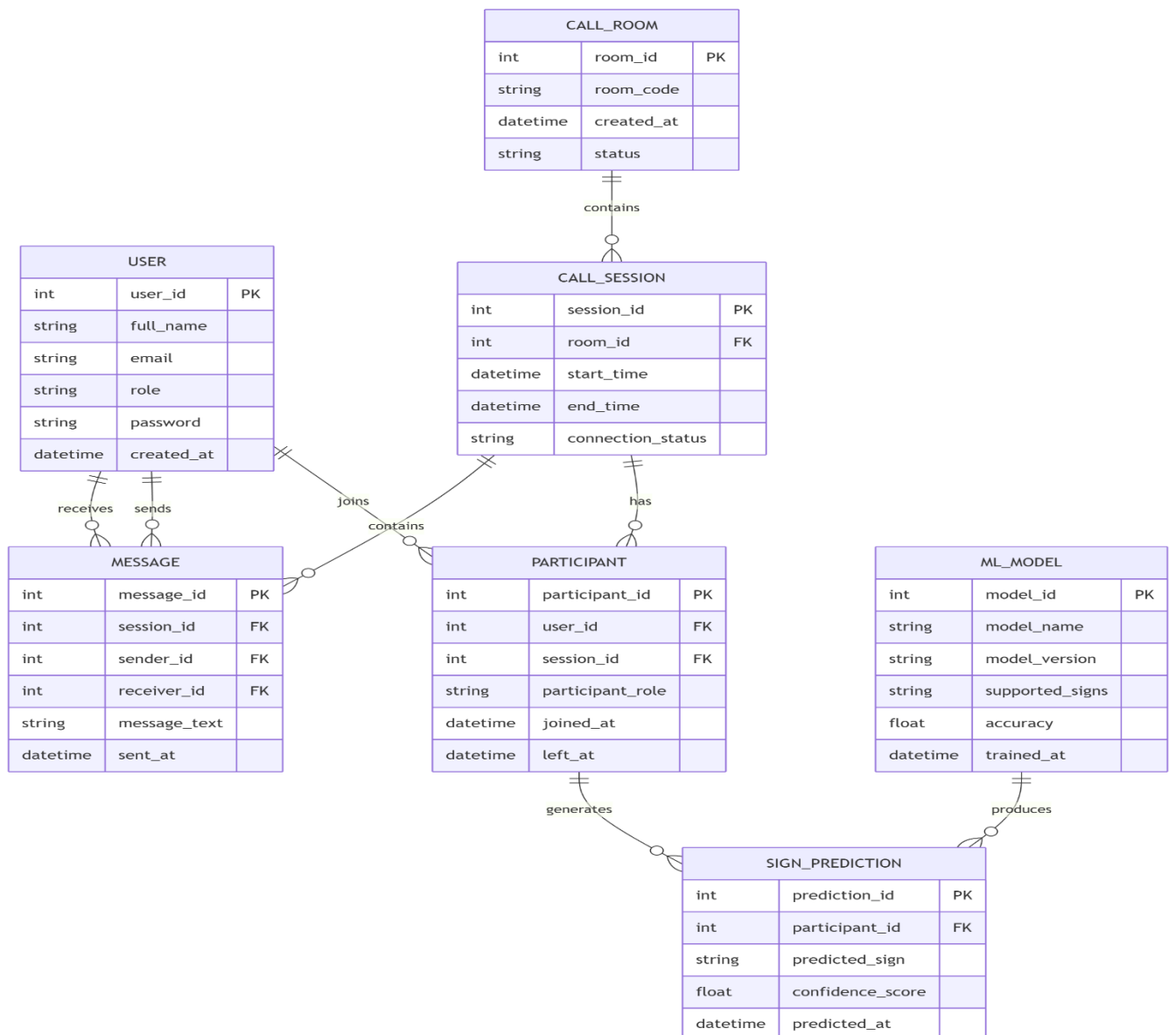


Figure 5: Entity Relationship Diagram

In the current version of the SignBridge system, there is no complex application database for storing users, call history, messages, or sign prediction records. The system mainly works in real time. Users join a room, communicate through WebRTC, send text messages through

WebSocket, and then leave the room. The messages and call sessions are not permanently stored in the database.

3.3. Sequence Diagram of Each SSD

In Chapter 2, SSDs showed interaction between the user and the system as a whole. In this section, sequence diagrams show the internal design flow between frontend, backend, WebSocket server, WebRTC, MediaPipe, and the ASL model.

We will create them one by one. First are **UC-01 to UC-03**.

3.3.1. Join Room Call

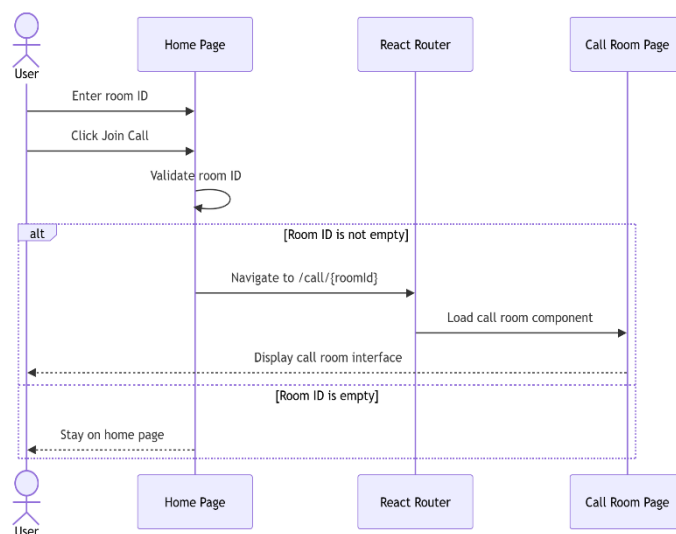


Figure 6:Seq Diagram Join Call Room

3.3.2. Start Video Communication

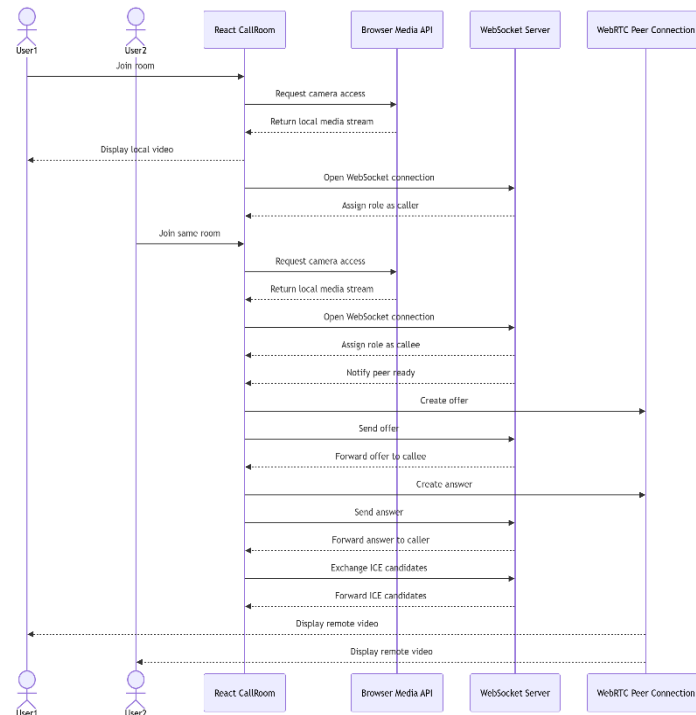


Figure 7: Start Video Communication

3.3.3. Detect Hand Landmarks

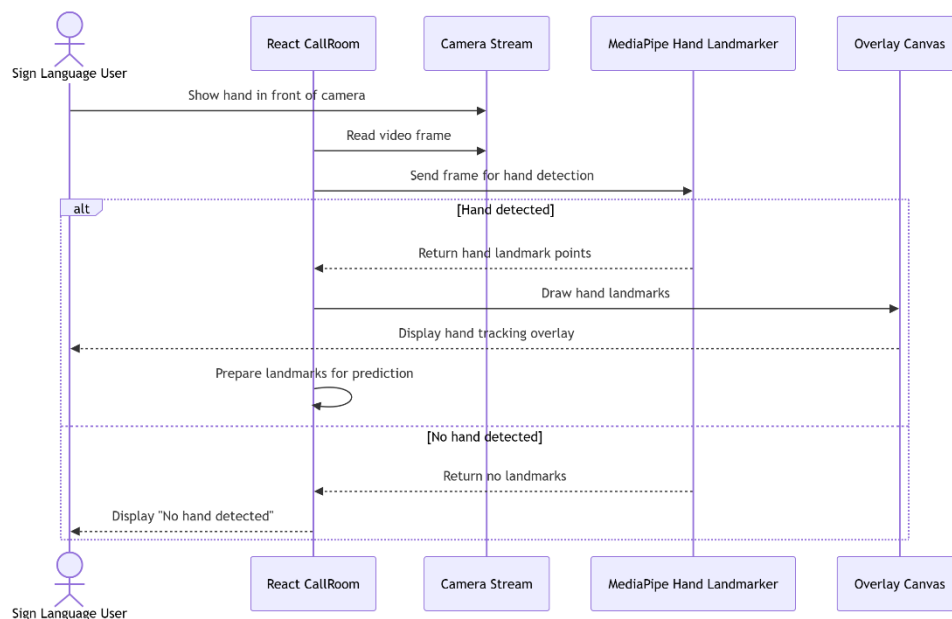


Figure 8: Detect Hand Landmarks

3.3.4. Recognize ASL Sign

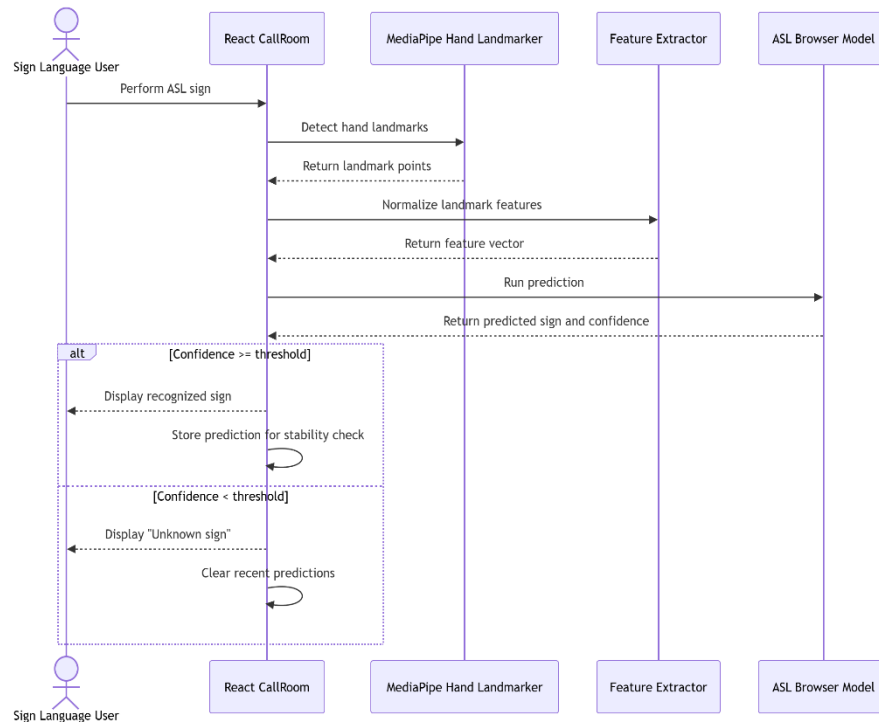


Figure 9: Recognize ASL

3.3.5. Generate Text From Signs

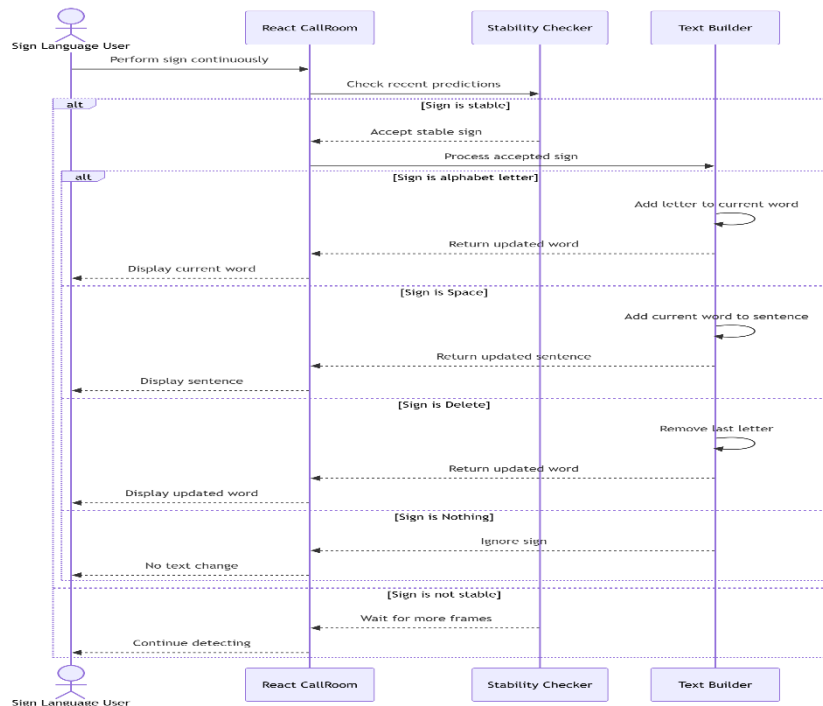


Figure 10: Generate Text From Sign

3.4. System Architecture Diagram

The SignBridge system follows a 3-tier architecture combined with peer-to-peer video communication and browser-based machine learning inference. The architecture separates the frontend, backend, and machine learning pipeline while allowing real-time communication between two users.

3.4.1. Architecture Overview

SignBridge consists of three main layers:

1. **Presentation Layer (Frontend)**

Built using React and Vite. Handles user interface, camera access, microphone access, WebRTC video calling, hand detection, ASL sign recognition, speech recognition, text generation, sign playback, and text-to-speech.

2. **Application Layer (Backend)**

Built using Django and Django Channels. Handles WebSocket signaling, room management, caller/callee role assignment, and message broadcasting between users.

3. **Machine Learning Layer**

Developed using Python, TensorFlow/Keras, MediaPipe, and OpenCV. Used for training the ASL model, extracting hand landmarks, evaluating accuracy, and exporting the browser model.

Video streams are transmitted directly between users using **WebRTC**, while the backend only handles signaling and message exchange. ASL sign prediction is performed in the browser, which reduces server load and improves response time

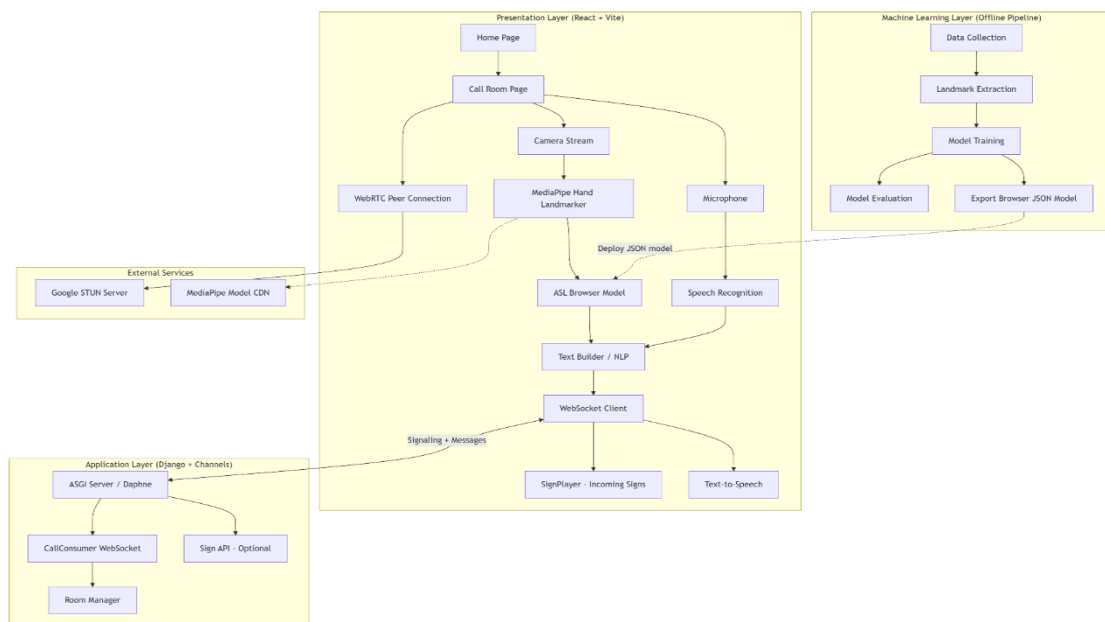


Figure 11: Architecture Overview

3.5. Database Design Details

Database design explains how data is stored, organized, and related in a system. In SignBridge, the current implementation is mainly **real-time** and does not heavily depend on persistent database storage. Most communication happens live through WebRTC and WebSockets. Messages, sign predictions, and call sessions are processed during the call and are not permanently saved in the current version.

However, Django uses **SQLite** as the default database for basic framework operations such as admin support and internal system tables. For documentation and future extension, this section describes both the **current database usage** and a **possible extended database design**.

3.5.1 Current Database Usage

In the present implementation, SignBridge uses SQLite with Django's default configuration. The database file is stored as db.sqlite3 in the backend folder.

Currently, the system does not store:

1. User accounts
2. Call history
3. Chat messages
4. Sign prediction logs
5. Video session records

The main reason is that SignBridge is designed as a real-time communication prototype. Users join a room, communicate, send messages, and leave the room without saving permanent records.

The database is mainly used for:

1. Django internal tables
2. Admin panel support
3. Future extensibility

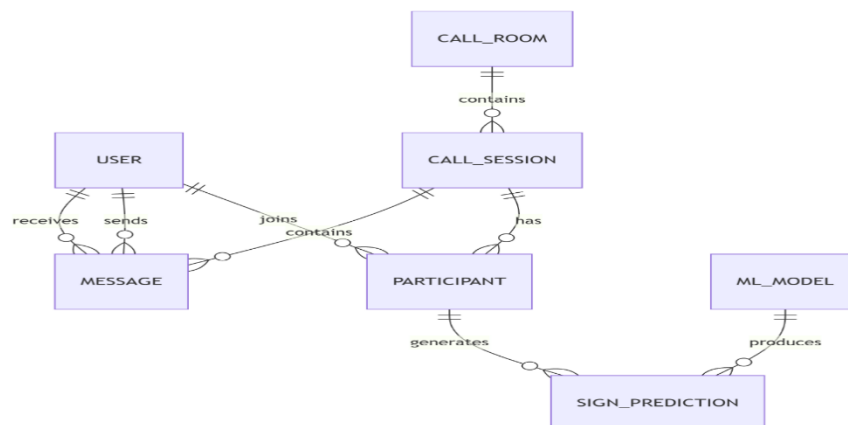


Figure 12: Current Database Usage details

3.5. UI/UX Mockups or Wireframes

UI/UX design explains how users interact with the SignBridge system through visual screens. The interface is designed to be simple, clean, and accessible so that both sign language users and non-sign language users can communicate easily during a video call.

The SignBridge application has two main screens:

1. **Home Page**
2. **Call Room Page**

The call room page is further divided into smaller UI sections such as call header, video area, sign playback area, text builder, speech input, guide panel, and conversation messages.

3.6.1. Home page wireframe

The home page is the entry point of the system. The user enters a room ID and joins a call.

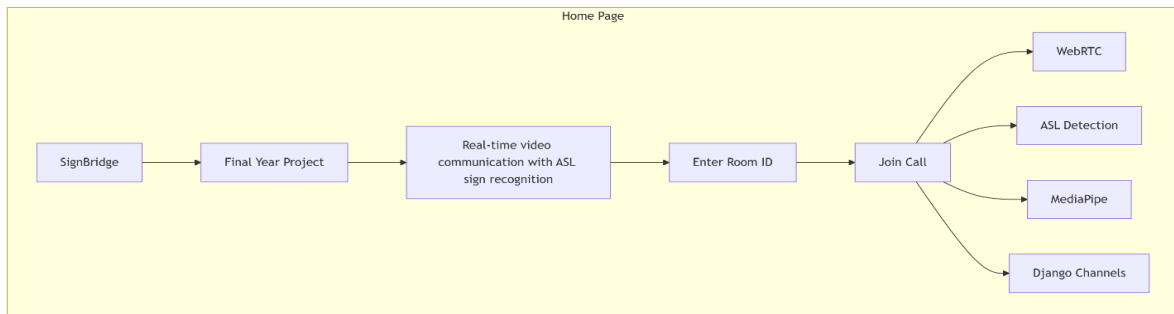


Figure 13: Home page wireframe

3.5.2. Call Room Page Wireframe

The call room page is the main working screen of the system. It contains video communication, sign detection, text generation, speech input, and message display.

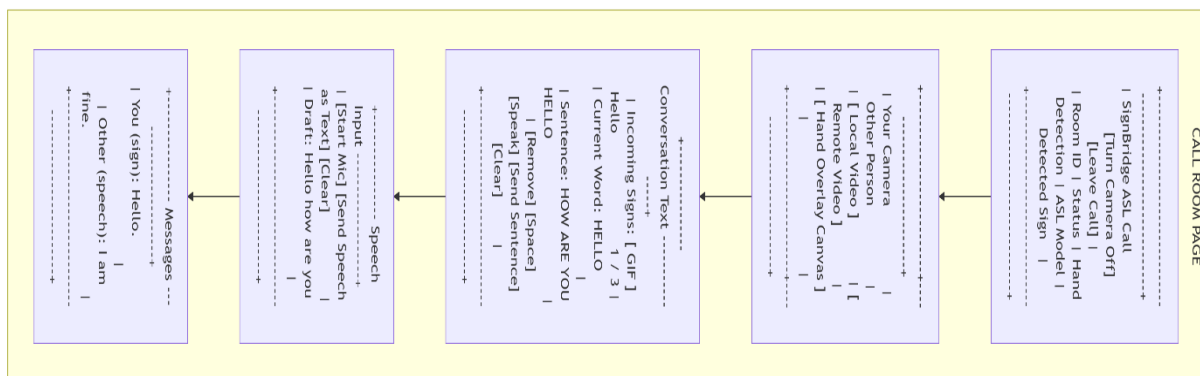


Figure 14: Call Room Page Wireframe

3.6.3. Call Header Section

Displays session information and main controls.

Components:

Application title

Room ID

Connection status

Hand detection status

ASL model status

Detected sign

Camera toggle button

Leave call button

Purpose: Gives the user real-time feedback about system status.

3.5.3. Video Grid Section

Shows live video streams of both participants.

Components:

- Local camera video
- Hand landmark overlay canvas
- Remote participant video

Purpose: Enables visual communication and hand sign detection.

3.5.4. Sign Player Section

SignPlayer Section

Displays ASL sign GIFs when the other person sends a spoken message.

Components:

- Incoming signs title
- Current sign label
- Sign GIF/image
- Progress indicator

Purpose: Helps the sign language user understand spoken messages through visual signs.

3.5.5. Word Builder Section

Used by the sign language user to build text from recognized signs.

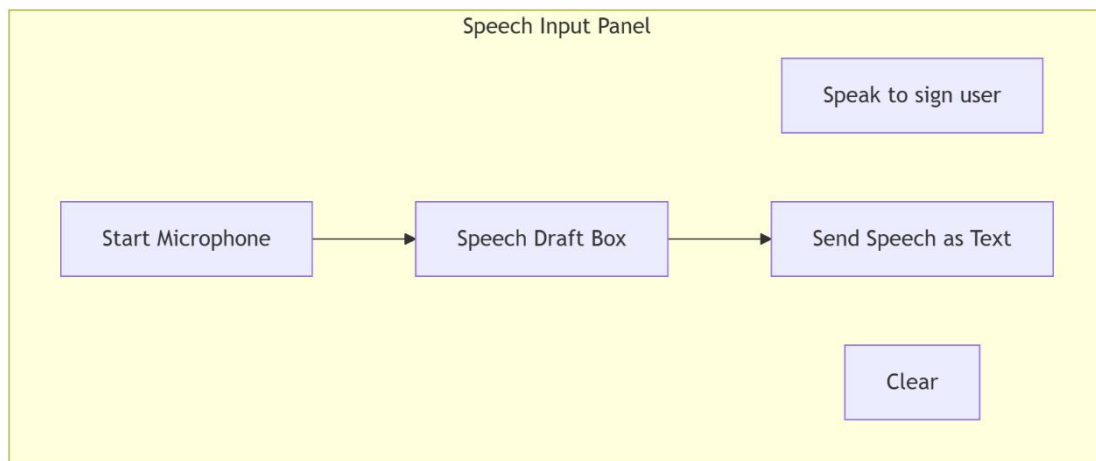
Components:

- Current word display
- Sentence preview
- Spacing status
- Remove last letter
- Add space
- Speak preview
- Send sentence
- Clear

Purpose: Converts recognized signs into readable text before sending.

3.5.6. Speech Input Section

Figure 15: Speech Input Section



Chapter 4

IMPLEMENTATION

This chapter describes how the SignBridge system was actually built. It explains the main algorithms, important code units, technologies used, system modules, and deployment process. The implementation follows the design described in Chapter 3 and converts the requirements from Chapter 1 into a working application.

4.1. Code / Algorithm of Main Scenarios and Main Units

4.1.1. Project Structure

The project is divided into three main folders:

- Frontend
- Backend
- ML

4.1.2. Join Call Room

Algorithm:

START

User opens home page

User enters room ID

IF room ID is empty THEN

Do nothing

ELSE

Navigate to /call/{roomId}

END IF

END

Implementation logic:

```
function joinRoom(event) {  
  event.preventDefault();  
  if (!roomId.trim()) return;  
  navigate(`/call/${roomId.trim()}`);  
}
```

4.1.3. Start Video Call

Algorithm:

START

Request camera and microphone access

Start local video stream

Create RTCPeerConnection

Connect to WebSocket ws://host:8000/ws/call/{roomId}/

Receive role: caller or callee

IF role is caller THEN

Wait for peer-ready

Create WebRTC offer

Send offer through WebSocket

ELSE IF role is callee THEN

Receive offer

Create answer

Send answer through WebSocket

END IF

Exchange ICE candidates

Display remote video

END

Backend room logic:

if len(CallConsumer.rooms[self.room_id]) >= 2:

send room-full message

close connection

self.role = "caller" if room empty else "callee"

4.1.4. Hand Landmark Detection

Algorithm:

START

Load MediaPipe HandLandmarker

WHILE camera is active DO


```

Read video frame
Detect hand landmarks
IF hand found THEN
    Draw landmarks on canvas overlay
    Send landmarks to ASL model
ELSE
    Show "No hand detected"
    Check for auto spacing pause
END IF
END WHILE
END

```

4.1.5. ASL Sign Recognition

Algorithm:

```

START
    Load ASL browser model JSON
    Extract 63 hand landmark features
    Pass features through dense neural network layers
    Apply ReLU on hidden layers
    Apply Softmax on output layer
    Select class with highest confidence
    IF confidence >= 0.82 THEN
        Add prediction to recent prediction window
        IF same sign appears STABLE_COUNT times THEN
            Accept sign and update current word
        END IF
    ELSE
        Show unknown sign
    END IF
END

```

4.1.6. Build and Send Sentence

Algorithm:

```

START

```

```

User performs stable signs
IF sign is alphabet THEN
    Add letter to current word
ELSE IF sign is space THEN
    Move current word to sentence
ELSE IF sign is delete THEN
    Remove last letter
ELSE IF no hand for pause duration THEN
    Auto-add current word to sentence
END IF

```

```

User clicks Send Sentence
Normalize sentence using NLP rules
Send message through WebSocket with source = "sign"
Display message locally
Speak message using browser TTS
Clear current word and sentence
END

```

4.1.7. Speech to Text to Signs

Algorithm:

```

START
    User clicks Start Microphone
    Browser Speech Recognition converts voice to text
    User clicks Send Speech as Text
    Normalize sentence
    Send message through WebSocket with source = "speech"
    Other user receives message
    Convert text into sign queue using sign dictionary
    SignPlayer displays supported sign GIFs one by one
END

```

Text-to-sign conversion logic:

1. Convert text to lowercase

2. Remove punctuation
3. Match multi-word phrases first (example: "thank you")
4. Match single words using dictionary
5. Create queue of supported sign GIFs
6. Display each sign for 2.8 seconds

4.1.8. WebSocket Message Broadcasting

Algorithm:

START

User connects to room WebSocket

IF room already has 2 users THEN

 Reject connection

ELSE

 Add user to room group

 Assign caller/callee role

END IF

WHEN message received THEN

 Broadcast message to all other users in same room

END WHEN

WHEN user disconnects THEN

 Remove user from room

 Notify other user: peer-left

END WHEN

END

4.1.9. Machine Learning Training Algorithm

Algorithm:

START

Load landmark dataset (x.npy, y.npy, labels.npy)

Split data into training and testing (80/20)

Build dense neural network

Train model for 40 epochs

Evaluate test accuracy

Save Keras model

Export JSON model for browser use

END

4.2. Tools and Technologies Used

4.2.1. Frontend Technologies

Technology	Version	Purpose
React	19.2.6	UI development
React DOM	19.2.6	Browser rendering
React Router DOM	7.15.0	Page routing
Vite	8.0.12	Frontend build tool
@mediapipe/tasks-vision	0.10.35	Hand landmark detection
ESLint	10.3.0	Code quality checking
WebRTC API	Browser built-in	Video communication
Web Speech API	Browser built-in	Speech recognition and TTS

Table 18:Frontend Technologies

4.2.2. Backend Technologies

Technology	Version	Purpose
Python	3.x	Backend language
Django	6.0.5	Web framework
Django Channels	Latest in venv	WebSocket support
Daphne	Latest in venv	ASGI server
Django REST Framework	3.17.1	REST API support
django-cors-headers	Latest in venv	Cross-origin requests
SQLite	Built-in	Default database

Table 19:Backend Technologies

4.2.3. Machine Learning Technologies

Technology	Purpose
Python	ML scripting
TensorFlow / Keras	Model training
NumPy	Numerical data handling
scikit-learn	Train/test split and evaluation
MediaPipe	Hand landmark extraction
OpenCV	Image and webcam processing

Table 20:ML Technologies

4.2.4. External Services

Service	Purpose
Google STUN Server	WebRTC connection support
jsDelivr CDN	MediaPipe WASM files
Google Cloud Storage	Hand landmarker model file

4.3. System Modules Description

4.3.1. Frontend Modules

Module 1: Routing Module

Files: App.jsx, main.jsx

Handles navigation between home page and call room.

Module 2: Home Module

File: Home.jsx

Provides room ID input and join call functionality.

Module 3: Call Room Module

File: CallRoom.jsx

Main controller of the application. It manages:

- Camera and microphone access
- WebRTC connection
- WebSocket signaling
- Hand detection loop
- ASL prediction
- Text building
- Message sending and receiving
- Sign playback trigger

Module 4: Video Module

File: VideoGrid.jsx

Displays local and remote video streams and hand overlay canvas.

Module 5: Transcript Module

File: TranscriptPanel.jsx

Combines word builder, speech input, sign player, guide panel, and message display.

Module 6: Speech Recognition Module

File: useSpeechRecognition.js

Converts spoken language into draft text using browser speech recognition.

Module 7: NLP Module

File: nlp.js

Normalizes generated sentences into readable English.

Module 8: Text-to-Sign Module

Files: textToSign.js, signDictionary.js, SignPlayer.jsx

Converts incoming spoken text into visual ASL sign GIFs.

Module 9: ASL Browser Model Module

File: asl_browser_model.json

Stores trained neural network weights for browser-side inference.

4.3.2. Backend Modules

Module 1: Django Configuration Module

Files: settings.py, urls.py, asgi.py

Configures apps, database, CORS, ASGI routing, and installed applications.

Module 2: WebSocket Signaling Module

Files: calls/consumers.py, calls/routing.py

Handles room joining, role assignment, message relay, and peer disconnect events.

Module 3: Optional Sign API Module

Files: signs/views.py, signs/predictor.py

Provides server-side sign prediction using TensorFlow and MediaPipe. This is optional because the main system performs inference in the browser.

4.3.4. Machine Learning Modules

Module 1: Dataset Processing

File: extract_asl_landmarks.py

Extracts 63-dimensional landmark features from ASL dataset images.

Module 2: Model Training

File: train_asl_browser_model.py

Trains dense neural network and exports browser JSON model.

Module 3: Model Evaluation

File: evaluate_asl_model.py

Generates classification report and confusion matrix.

Module 4: Live Testing

File: predict_asl_live.py

Tests trained model using webcam in Python/OpenCV environment.

Module 5: Model Conversion

Files: convert_asl_to_tflite.py, convert_asl_to_onnx.py

Supports exporting model to other formats for future deployment.

4.4. Deployment Details

Development Environment Requirements

Hardware:

- Laptop/Desktop with webcam and microphone
- Minimum 8 GB RAM recommended
- Internet connection for CDN model loading

Software:

- Node.js and npm
- Python 3.x
- Modern browser (Chrome or Edge recommended)
- Git optional

4.4.1. Project Setup Steps

Setup Backend

```
cd backend
```

```
python -m venv venv
```

```
venv\Scripts\activate
```

```
pip install django channels daphne django-cors-headers djangorestframework
```

```
python manage.py migrate
```

```
python manage.py runserver
```

Setup Frontend

```
cd frontend
```

```
npm install
```

```
npm run dev
```

Setup ML Environment

```
cd ml
```

```
python -m venv venv
```

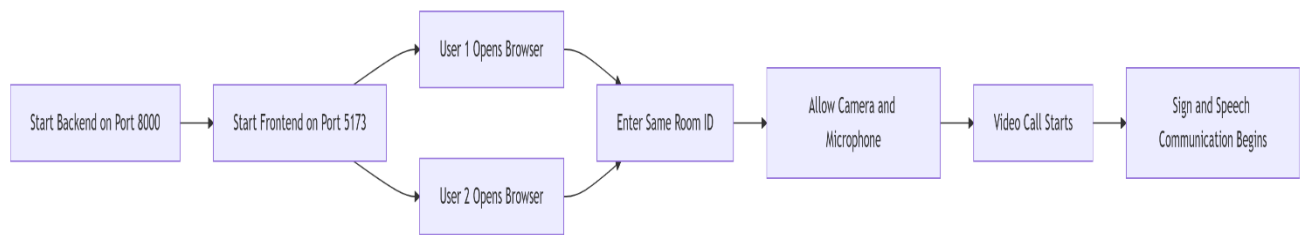
```
venv\Scripts\activate
```

```
pip install tensorflow numpy scikit-learn mediapipe opencv-python
```

```
python extract_asl_landmarks.py
```

python train_asl_browser_model.py

python evaluate_asl_model.py



4.4.2. Deployment Architecture

Component	Port	Role
Frontend (Vite)	5173	UI and browser inference
Backend (Django Channels)	8000	WebSocket signaling
WebRTC	Browser P2P	Video streaming
SQLite	Local file	Django default storage

4.4.3. Deployment Limitations

The current deployment is suitable for development and demonstration, not full production:

- Uses in-memory channel layer
- Debug mode enabled
- CORS open for all origins
- No HTTPS enforced
- No persistent message storage
- Two users only per room

For production deployment, the system would need:

- HTTPS and secure WebSockets
- Redis channel layer
- Environment-based secret keys
- Cloud hosting
- Proper .gitignore for venv and node_modules

Chapter 5

System Testing

System testing verifies that SignBridge works correctly according to the requirements defined in Chapter 1. Testing ensures that video calling, sign recognition, text generation, speech input, sign playback, and message exchange function properly individually and together.

This chapter includes test cases, different testing types, and a test summary report.

5.1. Test Case

The following test cases were designed based on the functional requirements of the SignBridge system.

5.1.1. Test Case Table

TC-01		Name: Join call with valid room ID	
System: SignBridge		SubSystem: Home	
Designed By:	Hammad Ali	Design Date:	June 2026
Executed By:	Hammad Ali	Execution Date:	June 2026
Description: Verify that entering a valid room ID navigates the user to the call room.			
Pre-Condition: System is running. Home module is active.			
Steps	1. Navigate to home page. 2. Enter valid room ID (room101). 3. Click Join Call button.		
Expected	User is navigated to the call room page.		
Actual	User navigated to call room successfully.		
Status	PASS		

Table 21:Join call with valid room ID

TC-02		Name: Join call with empty room ID	
System: SignBridge		SubSystem: Home	
Designed By:	Hammad Ali	Design Date:	June 2026
Executed By:	Hammad Ali	Execution Date:	June 2026
Description: Verify that an empty room ID prevents navigation.			
Pre-Condition: System is running. Home module is active.			
Steps	1. Navigate to home page. 2. Leave room ID field empty. 3. Click Join Call button.		
Expected	System stays on home page. No navigation occurs.		
Actual	No navigation occurred. System remained on home page.		
Status	PASS		

Table 22:Join call with empty room ID

TC-03		Name: Camera permission granted	
System: SignBridge		SubSystem: Call Room	
Designed By:	Hammad Ali	Design Date:	June 2026
Executed By:	Hammad Ali	Execution Date:	June 2026
Description: Verify that granting camera/mic permission starts local video.			
Pre-Condition: System is running. Call Room module is active.			
Steps	1. Navigate to call room. 2. When prompted, click Allow for camera and microphone.		

	3. Verify local video displayed.
Expected	Local video stream starts and is displayed.
Actual	Local video displayed after permission granted.
Status	PASS

Table 23: Camera permission granted

TC-04		Name: Camera permission denied	
System: SignBridge		SubSystem: Call Room	
Designed By:	Hammad Ali	Design Date:	June 2026
Executed By:	Hammad Ali	Execution Date:	June 2026
Description: Verify that denying camera permission shows an error and prevents the call.			
Pre-Condition: System is running. Call Room module is active.			
Steps	1. Navigate to call room. 2. When prompted, click Deny for camera permission. 3. Verify error/status shown.		
Expected	Error or status message shown. Call cannot start properly.		
Actual	Error shown. Call did not start.		
Status	PASS		

Table 24: Camera permission Rejected

TC-05			Name: Connect two users in same room	
System: SignBridge			SubSystem: Call Room	
Designed By:		Hammad Ali	Design Date:	June 2026

Executed By:		Hammad Ali	Execution Date:	June 2026
	Description: Verify that two users joining the same room can see each other via WebRTC.			
	Pre-Condition: Backend server running on port 8000. Frontend running on port 5173. Two devices/browsers available.			
Steps		1. Open SignBridge on Browser 1. 2. Enter room ID testroom and click Join Call. 3. Allow camera and microphone. 4. Open SignBridge on Browser 2. 5. Enter same room ID testroom and click Join Call. 6. Allow camera and microphone. 7. Verify both users see each other's video.		
Expected		Both users connect and see each other's remote video.		
Actual		Both users connected. Remote video visible on both screens.		
Status		PASS		

Table 25: Connect two users in same room

TC-06		Name: Third user joins room	
System: SignBridge		SubSystem: Call Room	
Designed By:	Hammad Ali	Design Date:	June 2026
Executed By:	Hammad Ali	Execution Date:	June 2026
Description: Verify that a third user is rejected when the room is full.			
Pre-Condition: System is running. Call Room module is active. Two users already connected.			
Steps	1. Open SignBridge on a third browser. 2. Enter the same room ID.		

	3. Click Join Call.
Expected	Connection rejected. Room full message shown.
Actual	Third user rejected. Room full message displayed.
Status	PASS

Table 26:Third user joins room

TC-07		Name: Hand shown to camera	
System: SignBridge		SubSystem: Hand Detection	
Designed By:	Hammad Ali	Design Date:	June 2026
Executed By:	Hammad Ali	Execution Date:	June 2026
Description: Verify that a visible hand is detected and landmarks are drawn.			
Pre-Condition: System is running. Hand Detection module is active.			
Steps	1. Enter call room with camera active. 2. Show hand clearly to the camera. 3. Observe overlay canvas.		
Expected	Hand landmarks drawn on overlay canvas.		
Actual	Hand overlay displayed with landmarks drawn correctly.		
Status	PASS		

Table 27:Hand shown to camera

5.1.2. Integration Testing

Integration testing verifies that multiple modules work together correctly.

Integration	Modules Combined	Result
Frontend + WebSocket	React CallRoom + Django Channels	Pass
WebRTC + Signaling	Peer connection + WebSocket messages	Pass
MediaPipe + ASL Model	Hand detection + sign prediction	Pass

Speech Input + WebSocket	Speech hook + message sending	Pass
Text-to-Sign + SignPlayer	textToSign.js + SignPlayer.jsx	Pass
ML Export + Frontend	JSON model + browser inference	Pass

5.1.3. Final Testing Conclusion

The Sign Bridge system successfully passed the planned test cases for its supported features. The application demonstrates:

- Real-time video communication
- ASL alphabet recognition
- Sign-to-text communication
- Speech-to-text communication
- Text-to-sign visual playback
- Text-to-speech output

Therefore, the system is considered **functionally correct, operationally usable, and ready for final project demonstration and documentation submission.**

Chapter 6

User Interface

6.1. Home Page

This figure shows the home page of the Sign Bridge application. This page serves as the entry point for users to join a real-time communication session. The interface provides a text field where users can enter a unique Room ID and a **Join Call** button to connect to an existing video communication room.

The page also highlights the core technologies used in the system, including **WebRTC** for peer-to-peer video communication, **ASL Detection** for sign language recognition, **MediaPipe** for hand tracking and landmark detection, and **Django Channels** for real-time signaling and communication management.

The user-friendly design allows participants to quickly access a communication room and start real-time video interaction with integrated sign language translation features.

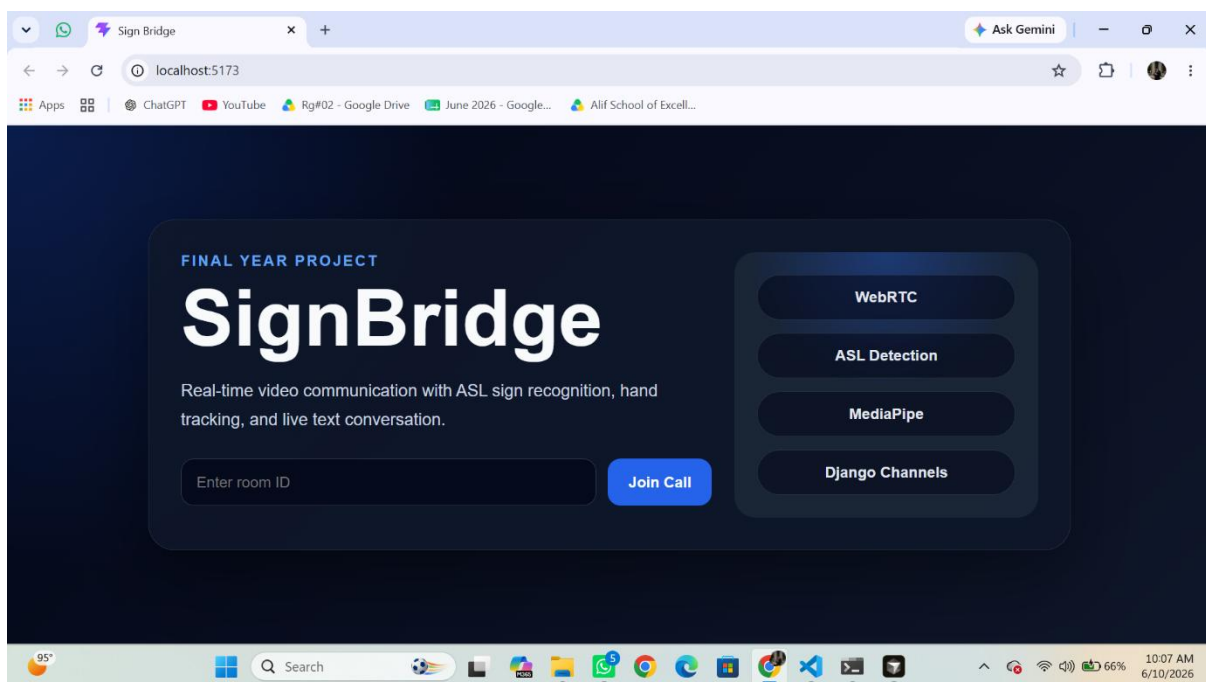


Figure 16:Home page

6.2. Video Call Interface

This figure shows the main video call interface of the SignBridge system after a user joins a communication room. This screen enables real-time communication between participants using WebRTC technology while integrating American Sign Language (ASL) recognition and text-based interaction.

The interface contains two video panels: **Your Camera**, which displays the local user's video stream, and **Other Person**, which displays the remote participant's video stream. At the top of the page, system status indicators provide information about the connection state, hand detection status, and ASL model readiness.

On the right side, the **Your Message** section displays the currently recognized sign and the complete message being constructed from detected gestures. Users can manage the generated text using the **Undo**, **Space**, **Send**, and **Clear** buttons. The **Chat** panel shows exchanged messages between participants during the call.

Additional features include **Speech Input** for converting spoken language into text and a **Text-to-Sign** section that presents spoken messages as sign language content to improve accessibility for deaf and hard-of-hearing users. Users can also control their camera and leave the call using the provided action buttons.

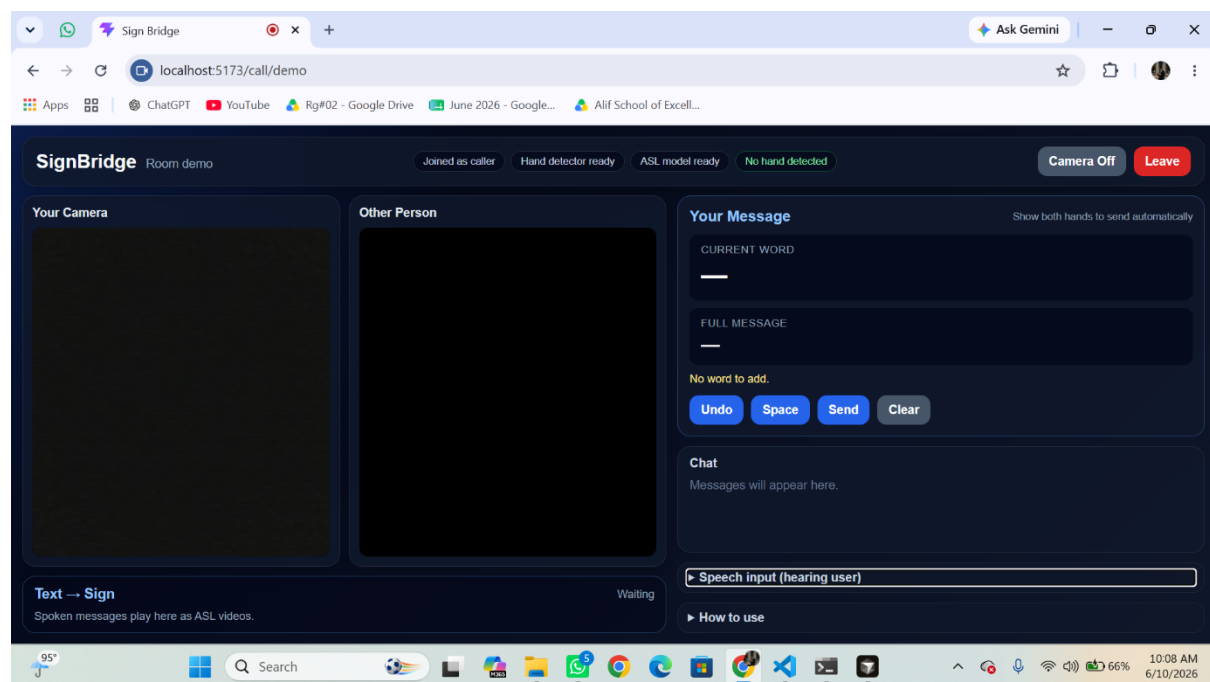
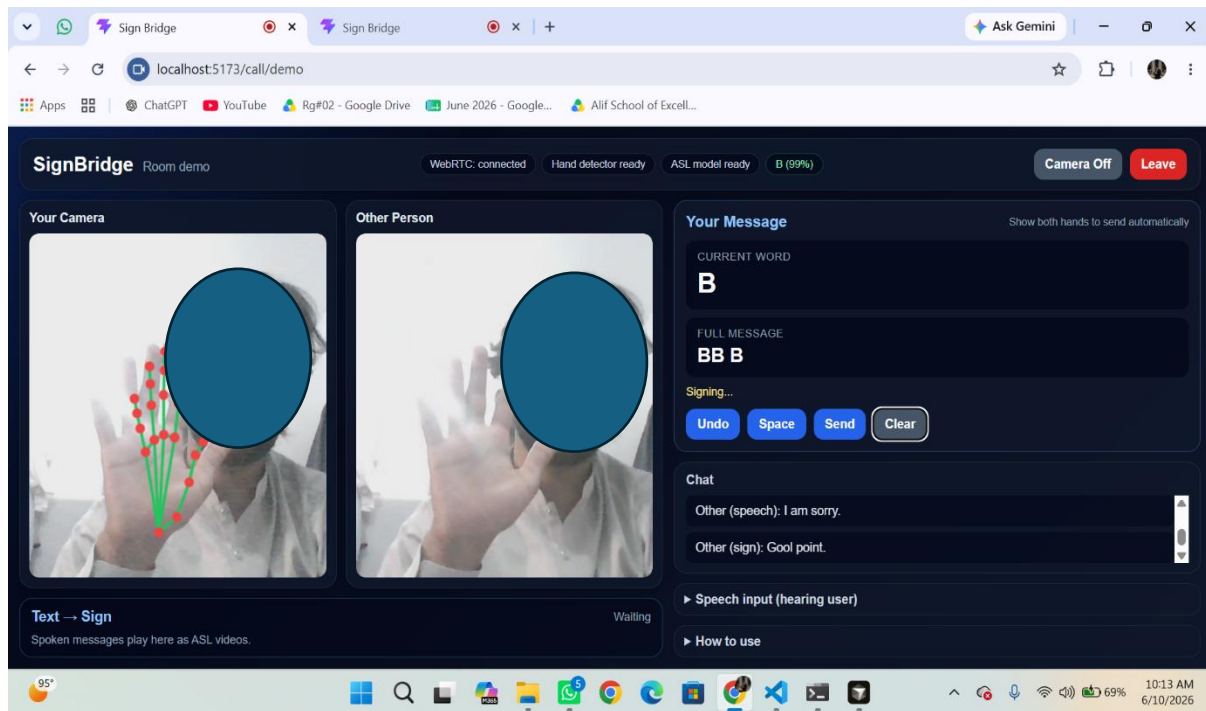


Figure 17: Video Call Interface

6.3. Detect Hand Landmarks

This shows the hand landmark detection interface used in the SignBridge system. This module utilizes MediaPipe to detect and track hand movements in real time through the user's webcam. The system identifies key hand landmarks and displays them as connected points and lines over the video feed.

Figure 18: Detect Hand Landmarks



6.3. Sign Recognition and message transfer

This shows the sign recognition and message transmission process in the SignBridge system. When a user performs a valid ASL gesture, the recognition model analyzes the detected hand landmarks and identifies the corresponding sign. The recognized sign is then converted into text and displayed in the message area.

Users can construct complete sentences by combining multiple recognized signs. Once the message is finalized, it can be transmitted to the other participant in the communication room. The receiving user instantly receives the translated text through the real-time chat system, enabling effective communication between sign language users and non-sign language users.

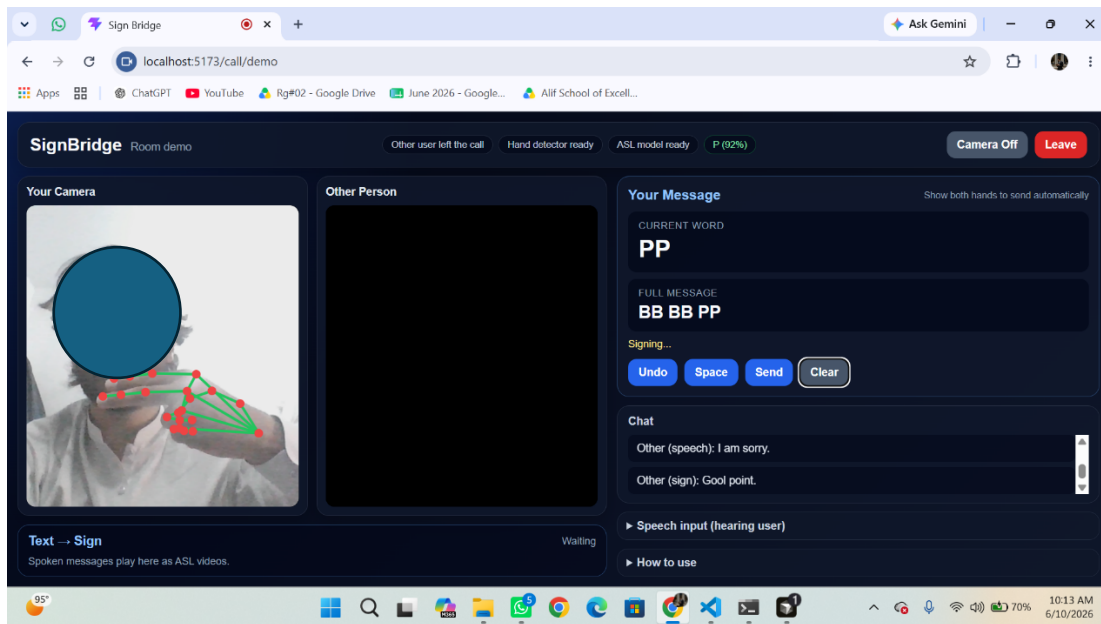


Figure 19: Sign Detection

6.4. Message Send

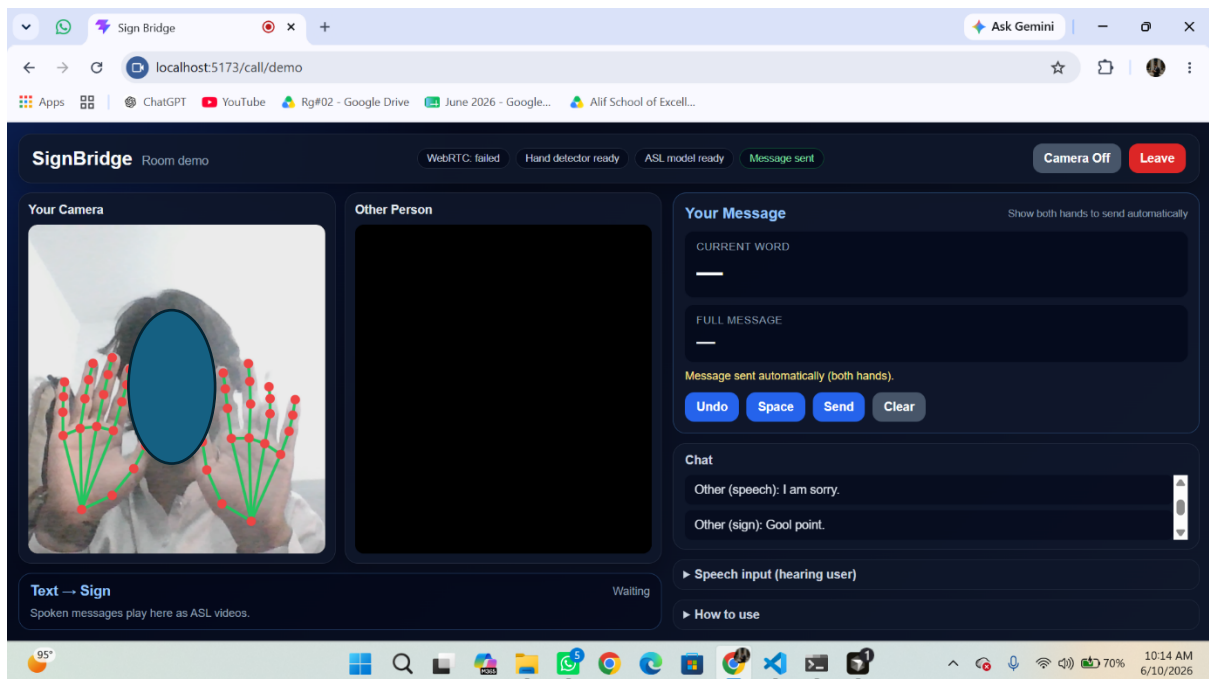


Figure 20: Message Send

6.5. Speech to Sign

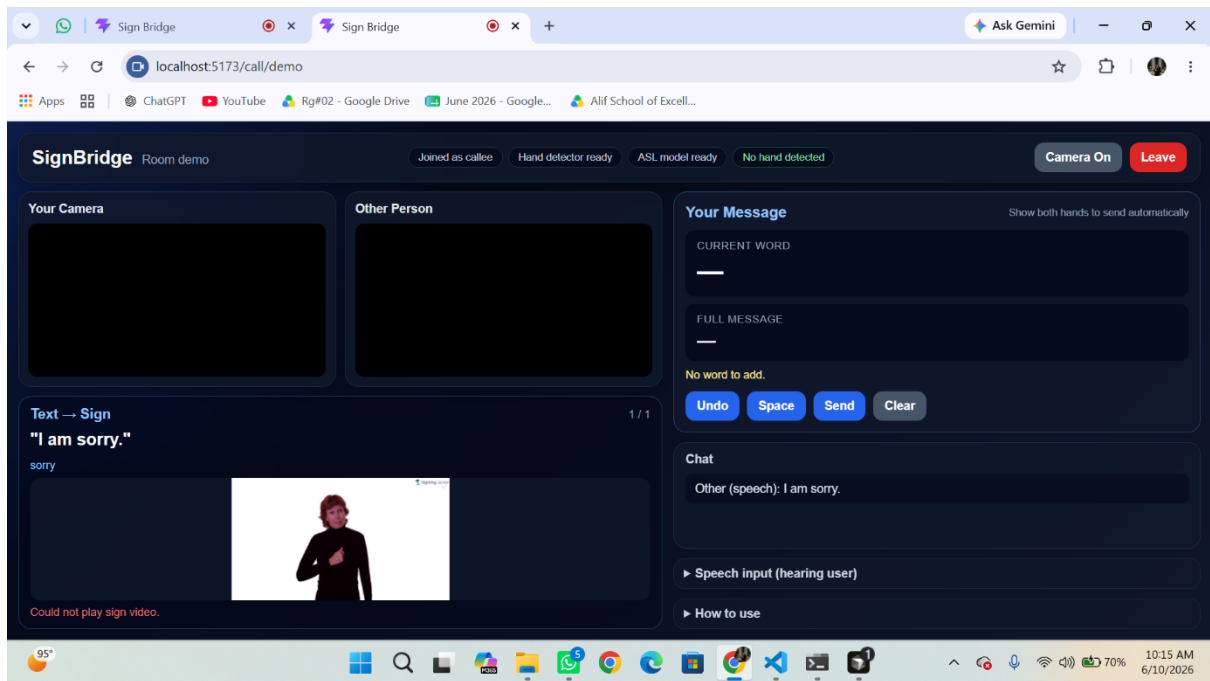


Figure 21: Convert Speech into sign